

The Blind Spot

How a vision-guided robot goes blind, and one line of code that notices

A complete account of the project: the algorithms from first principles, every number that decided something, the ROS 2 stack, and the mistakes, in order.

github.com/SaurabhKhimesra/blind-spot

Contents

1. What this is about	5
2. Where things are: frames and transforms	7
3. How a camera turns the world into numbers	10
4. Matrices, least squares, and the one idea that explains every failure	12
5. Feedback control in one chapter	15
6. The servo loop	19
7. Deriving the interaction matrix	21
8. The control law, and turning it into motion	24
9. The harness: what is actually run	26
10. Failure 1: camera retreat	31
11. Failure 2: feature dropout	33
12. Failure 3: two features left	35
13. Failure 4: the collapsed target	37
14. The 2001 partition	42
15. Truncation: declining to move blind	45
16. The switch: deciding per step	47
17. The guard that shipped	50
18. How correctness was established	55
19. Dead claims	58
20. A real camera and a real detector	61
21. ROS 2, from zero	66
22. Packaging it: the library and the ROS 2 node	69
23. Building the Gazebo cell	72
24. Kinematics and perception on the arm	76
25. What the arm found	79
26. The folding part, and the clip	82
27. Limits, collected	87
28. Telling the story	90
29. Glossary, and every number in one place	95

How to read this

Every chapter opens **in plain words**, with no jargon and no symbols, then gives the precise version with the measured numbers, and closes with **Say it like this** - a spoken version you could use in an interview.

The book assumes nothing. Part 0 builds frames, cameras, matrices and feedback from scratch; every term is defined where it first appears, and chapter 29 is a glossary plus every number in the book with what it means.

Never seen robotics: chapters 1 to 5 in order, then 10, 14, 17, then 26.

Know control, new to visual servoing: 6 to 9, then Part 2, then 17.

Preparing for an interview tomorrow: 28, then 19, then 27, then 23.

Reviewing the engineering: 9, 18, 19, 27.

Every number quoted here is produced by a script in the repository and, where it decided something, locked by a regression test. Where a claim of mine turned out to be wrong, the retraction is in the book next to the claim rather than deleted - chapter 19 is nothing else.

Part 0

Foundations

1. What this is about

The situation

A robot arm has a camera bolted to its wrist. On the part it is working on there are six small markers. The robot's job is to line itself up with that part — to plug something in, fit a panel, pick it up — and it does that by watching the markers and moving until they sit where they are supposed to sit in the picture.

This is real and ordinary. It is how a lot of vision-guided robots work, and the method has a name: **image-based visual servoing**, IBVS for short. "Servoing" just means continuously correcting toward a target.

The thing that goes wrong

The controller needs to know how far away the part is, and a single camera cannot measure distance directly. So the well-known fix, published in 2001, uses a substitute: **how big the patch covered by the markers looks**. Big patch, close. Small patch, far. Cheap, fast, works.

Now suppose the part folds, or flexes, or is seen from a steep angle. The patch gets smaller for a reason that has nothing to do with distance. The controller cannot tell the difference. It concludes "I am too far away" and drives toward the part — fast, confidently, and wrongly.

An everyday version: judge distance by how big a sheet of paper looks. Someone folds the paper in half. It looks smaller, so you step toward it. The paper never moved.

In a factory, that step forward is a bent part, a scrapped assembly, or a safety stop that shuts down the line until someone walks over and resets it.

The question this project asks

Not "does the robot recover eventually" — it usually does. The question is what it does in the **half second** while its information is wrong, because that is when the damage happens.

The answer, measured: it speeds up rather than slowing down. In one benchmark, losing four of six markers makes the standard fix command a motion **288 times** larger than it was moving before. That is not a subtle numerical issue. That is a robot lunging.

What was built

Three things, in order:

1. **A study.** All four distinct ways this kind of controller goes blind, each reproduced in a small simulation with exact numbers, and every candidate fix compared on the measure that suits each failure.
2. **A guard.** One runtime check: watch the very measurement the controller leans on, compare it against what that measurement looks like on a known-good part, and when

it drifts out of range, switch the robot to a more conservative control law for as long as it takes. It closes the worst gap from 11.6× to **0.5×** — the robot ends up moving *slower* during the disruption than before it, which is the behaviour you want.

3. **A robot.** The guard packaged as a ROS 2 node, and a full simulated UR5e cell in Gazebo where the loop runs closed: camera → detector → guard → control law → the arm's own kinematics, ten times a second.

What was disproved, including by me

The project started from the assumption that choosing the control law each step would need machine learning. It does not. A hand-written rule beat the bar that was set in advance, by four times, and the learning plan was dropped.

Six further claims of mine died the same way, each with the measurement that killed it, and they are all in chapter 19 — including one where I attributed a velocity spike to my own guard, wrote it down, and had to retract it the next day when the measurement did not reproduce.

That chapter is the honest core of the project. A result is only as trustworthy as the number of chances it had to be wrong.

How this book is organised

- **Part 0 (chapters 2-5)** builds the background from nothing: frames and poses, how a camera turns the world into numbers, what a matrix inverse really does, and what feedback control is.
- **Part 1 (6-9)** derives visual servoing itself, including the interaction matrix, line by line.
- **Part 2 (10-13)** is the four failures, one chapter each.
- **Part 3 (14-17)** is the fixes: the 2001 partition, truncation, the runtime switch, and the guard that shipped.
- **Part 4 (18-20)** is how any of it was made trustworthy: finite differences, the test suite, the dead claims, and a rendered camera with a real detector.
- **Part 5 (21-26)** is the robot: ROS 2 from zero, the node, the Gazebo cell, kinematics, and the two findings that only appeared on an arm.
- **Part 6 (27-29)** is the limits, how to explain the work out loud, and a glossary.

Every chapter opens in plain language and ends with a spoken version you could say in an interview. Numbers are quoted exactly as the repository reports them, because a number you cannot reproduce is an opinion.

Say it like this

A camera-guided robot judges distance by how big the marker pattern looks. When the part folds or tilts, that pattern shrinks for reasons that have nothing to do with distance, and the controller drives at the part. I measured the four ways this class of controller goes blind, built a one-line runtime check that catches the case that matters, and put it on a simulated UR5e in ROS 2 and Gazebo to see what a real arm changes.

2. Where things are: frames and transforms

Everything later is about a camera and a part, so we need a precise way to say where something is. This chapter builds that from scratch. If you know rotation matrices already, skim to the last section, which fixes the conventions this project uses.

A position is three numbers, relative to something

"The marker is at (0.05, 0, 0)" means nothing on its own. Relative to what?

A **frame** is a fixed origin plus three directions (x, y, z) that everything is measured against. A position is three numbers **in a stated frame**. Change the frame and the numbers change, while the marker has not moved at all.

This project uses three frames constantly:

- the **object frame**: attached to the part. The six markers have fixed coordinates here, and they never change.
- the **camera frame**: attached to the camera. **z** points out of the lens, **x** right, **y** down. This is the computer-vision convention and it is what the code means by "the camera's view".
- the **world frame**: the robot cell. Only the arm chapters need it.

Rotation as a matrix

Turning something in 3D can be written as multiplying by a 3×3 matrix **R**. Rotating by angle θ about the z axis:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Check it on a simple case. Take the point (1, 0, 0) and $\theta = 90^\circ$. Then $\cos \theta = 0$, $\sin \theta = 1$, and the product is (0, 1, 0): the point on the x axis has swung onto the y axis. That is what a 90° turn about z should do.

Rotation matrices have two properties worth remembering, because the code relies on both:

- **The inverse is the transpose.** $R^{-1} = R^T$. Undoing a rotation costs nothing.
- **They compose by multiplication, right to left.** $R_1 R_2$ means "do R_2 first, then R_1 ". Order matters: turning right then up is not the same as turning up then right.

Position and rotation together: the 4×4

A rigid body has both an orientation and a position, so you keep both in one 4×4 matrix:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \quad \begin{array}{l} R = 3 \times 3 \text{ rotation} \\ t = 3 \times 1 \text{ position} \end{array}$$

The trick that makes this worth doing: to move a point p from one frame to another you write it as $(x, y, z, 1)$ and multiply. The bottom row of ones is what lets a single matrix multiplication do "rotate **and** shift", which a 3×3 cannot.

$$p_{\text{new}} = T \cdot [p; 1] \quad \rightarrow \quad p_{\text{new}} = R \cdot p + t$$

Two of these compose the same way rotations do: $T_{ac} = T_{ab} \cdot T_{bc}$. This is the whole of `make_pose`, `transform_points` and the pose bookkeeping in `ibvs_core.py`.

The notation used throughout the repo is `cTo`: the pose of the **o**bject expressed in the **c**amera frame. Read it right to left — "object, into camera". So `transform_points(cTo, P)` takes marker coordinates written in the object frame and returns where they sit in the camera's frame.

Velocity: the twist

A moving rigid body has a linear velocity and an angular velocity. Stack them into six numbers and call it a **twist**:

$$v = (v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)$$

The first three are metres per second, the last three radians per second. Every control law in this project outputs exactly this: six numbers, the camera's commanded twist.

If a point sits at p in the camera frame and the *camera* moves with twist v , the point's coordinates in the camera frame change as

$$\dot{p} = -v_{\text{linear}} - \omega \times p$$

Both minus signs are there because the camera moves, not the point: if the camera slides right, everything in its frame slides left. This equation is the seed of the entire project — chapter 7 differentiates the camera projection using exactly this, and out falls the interaction matrix.

From twist to pose: the matrix exponential

A twist says how fast; a pose says where. To apply a twist for a short time dt you need the pose update. For rotations you cannot just add — adding angles works in 2D, not in 3D — so the correct operation is the **matrix exponential**, written `se3_exp`:

$$cTo \leftarrow \text{se3_exp}(-v \cdot dt) \cdot cTo$$

Think of it as "follow this constant velocity for dt seconds, exactly, along the curved path rotations actually take". Its inverse, `se3_log`, turns a pose difference back into the twist

that would produce it. Both live in `ibvs_core.py` and both are verified by finite difference (chapter 18), because a sign slip here produces a controller that behaves oddly in a way you will wrongly blame on the control law.

The minus sign in front of `v` is the same bookkeeping as before: the twist describes the camera's motion, and `cTo` describes where the object is *as seen from* the camera.

Say it like this

Positions are three numbers in a named frame; poses are a rotation and a translation kept in one 4×4 so a single multiply moves a point between frames. Velocities are six numbers, a twist. Applying a twist to a pose uses the matrix exponential rather than addition, because rotations do not add.

3. How a camera turns the world into numbers

The pinhole model, derived

Imagine a box with a tiny hole in one face and film on the opposite face. Light from a point in the world passes through the hole and lands on the film. That is a **pinhole camera**, and it is the model every equation here uses.

Put the hole at the origin of the camera frame, with z pointing out at the scene, and the image plane at distance f in front of it. A point sits at (X, Y, Z) . The ray from the point to the hole hits the image plane at some (u, v) .

Similar triangles do the rest. The triangle from the origin out to the point has "height" X at "depth" Z . The triangle from the origin to the image plane has height u at depth f . They are the same shape, so

$$\begin{array}{ll} u / f = X / Z & \rightarrow u = f \cdot X / Z \\ v / f = Y / Z & \rightarrow v = f \cdot Y / Z \end{array}$$

That is the whole model. Two consequences drive everything later:

- **Everything is divided by depth.** Twice as far away means half the size.
- **Depth is not recoverable from one point.** A point twice as far and twice as big lands in exactly the same place. One camera, one point, no depth.

Pixels, and why this project does not use them

Real sensors count pixels, not metres, and the optical axis rarely lands exactly at the middle pixel. So the practical version is

$$\begin{array}{l} u_{px} = f_x \cdot (X/Z) + c_x \\ v_{px} = f_y \cdot (Y/Z) + c_y \end{array}$$

where f_x , f_y are focal lengths in pixels and (c_x, c_y) is the **principal point**, where the optical axis pierces the sensor. Those four numbers are the camera **intrinsic**s, and in ROS they arrive in a `sensor_msgs/CameraInfo` message.

Divide them back out and you get **normalised image coordinates**:

$$\begin{array}{l} x = (u_{px} - c_x) / f_x = X / Z \\ y = (v_{px} - c_y) / f_y = Y / Z \end{array}$$

This project works entirely in x , y . The reason is that they are the geometry, with the particular sensor divided out — the same target at the same pose gives the same x , y on any camera. That is why `blindspot/units.py` exists, why it *requires* the principal point rather than assuming the middle of the image, and why the ROS node insists on `camera_info`.

That "assuming the middle" is not a hypothetical. Chapter 20 is the story of half a pixel: OpenGL puts the optical axis at $w/2$, OpenCV indexes pixel centres so the right answer is $(w-1)/2$, and the resulting bias of $(-0.4904, -0.5010)$ px was hidden for a while by a corner-refinement method whose own convention cancelled it.

What the camera can and cannot tell you

For one point you get two numbers, x and y . The robot has six degrees of freedom. So:

- **One point** gives 2 constraints. Not enough.
- **Three points** give 6 constraints for 6 unknowns. Just enough, in general.
- **Six points** give 12 constraints for 6 unknowns, comfortably redundant — which is why the targets here have six markers and why losing half of them is survivable while losing four is not (chapter 12).

"In general" is doing real work in that sentence. Some arrangements of points carry less information than their count suggests: three points **in a line** are worth less than three points in a triangle, and there is a specific surface in space — the **danger cylinder** through three points — where the geometry is singular no matter how good your camera is. Those cases are in chapters 13 and 27, and they are the reason this project measures *geometry* rather than counting features.

Depth, the quantity nobody has

The projection divides by Z , so any controller that reasons about metres needs Z . Options in practice:

1. A depth sensor. Adds hardware and its own failure modes.
2. Estimate it from the target's known size. Needs a model.
3. Use the depth you *expect* at the goal, and accept the error while you are far from it. This is standard and it is what the arm demo does (a constant 0.26 m).
4. Avoid needing it: replace the depth-dependent degree of freedom with something you can measure straight off the image. That is exactly what the 2001 partition does with the polygon area (chapter 14), and its weakness — the area can shrink for reasons that are not distance — is the subject of this entire book.

Say it like this

A pinhole camera divides everything by depth: a point at (X, Y, Z) lands at $(X/Z, Y/Z)$ in normalised coordinates, which are pixels with the intrinsics divided out. That division is why one camera cannot see depth from one point and why every controller here needs either a depth estimate or a substitute for it — and the substitute is what fails.

4. Matrices, least squares, and the one idea that explains every failure

This chapter is the mathematical spine of the book. It builds up to the **singular value decomposition**, because "a small singular value" is the answer to almost every "why did the robot do that?" in later chapters.

A matrix is a machine that turns one list of numbers into another

If v is six numbers (a camera twist) and L is a table with 12 rows and 6 columns, then $L v$ is 12 numbers (how fast each of six markers moves in the image, x and y each). That is all a matrix-vector product is: each output is a weighted sum of the inputs, and the weights are one row of the table.

The reverse question is the interesting one. **Given the output I want, what input produces it?** That is solving $L v = b$ for v , and three cases exist.

Three cases of solving

Square and healthy. As many equations as unknowns, all independent. One exact answer, $v = L^{-1} b$.

More equations than unknowns (12 rows, 6 unknowns — six markers). Usually no exact answer exists, because real measurements disagree slightly. The sensible answer is the one that misses by the least, measured as the sum of squared errors: **least squares**. Redundancy is protective here, because no single noisy measurement can drag the answer far.

Fewer equations than unknowns (4 rows, 6 unknowns — two markers). Now infinitely many answers fit exactly, and you must pick one. The standard choice is the **smallest** one, the minimum-norm solution.

The **pseudo-inverse**, written L^+ (`numpy.linalg.pinv`), does the right thing in all three cases: exact when it can, least squares when over-determined, minimum-norm when under-determined.

That last property matters more than it sounds. In chapter 12, classic IBVS survives having only two markers **precisely because** it is under-determined and the pseudo-inverse hands it the smallest solution, which is gentle. The 2001 partition, in the same situation, has an exactly-determined 4×4 with no slack, and it explodes.

The singular value decomposition

Any matrix L can be written

$$L = U \Sigma V^T$$

Read it right to left as three simple operations:

1. V^T — rotate the input space so its axes line up with the matrix's natural "input directions".
2. Σ — stretch each of those axes by a number: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_6 \geq 0$. These are the **singular values**.
3. U — rotate into the output space.

So a matrix does nothing more exotic than: rotate, stretch each axis, rotate. The singular values say **how strongly the matrix responds** along each of its six input directions.

For our L , the six input directions are combinations of camera motions, and the stretch says how much the image changes when the camera moves that way.

- A **large** singular value: move this way and the image changes a lot. Easy to see, easy to control.
- A **small** singular value: move this way and the image barely changes. The camera can move a long way with almost no evidence in the picture.

Why small singular values are dangerous

Inverting a matrix inverts the stretches: $1/\sigma$. So a direction with $\sigma = 0.004$ gets multiplied by 250 on the way back.

Now put the control law next to it. The controller says "I want the image to change like this", inverts, and commands the motion. If the requested change has even a *tiny* component along a weak direction — from noise, or from an asymmetry, or from a modelling error — the inverse amplifies it by $1/\sigma$ and the robot makes a large, confident, wrong movement.

That single sentence explains the velocity spikes in chapters 11, 12 and 13, the orbit in chapter 25, and the creep in chapter 26.

Vocabulary that follows from this:

- **Rank**: how many singular values are meaningfully non-zero, i.e. how many directions the matrix can actually see.
- **Null space**: directions with $\sigma = 0$. The matrix is completely blind to them; motion along them changes the image not at all.
- **Condition number**: $\sigma_1 / \sigma_{\text{last}}$. How lopsided the matrix is. Large means trouble.
- **Ill-conditioned**: a polite way of saying "some direction is nearly invisible and the inverse is about to shout".

Two ways to respond

Truncation. Look at the singular values, and any direction weaker than some fraction τ of the strongest, simply refuse to move along it. You give up control of a direction you could not see anyway, in exchange for not being amplified by $1/\sigma$. Chapter 15.

Watching the numbers. Compute a singular value each step and treat it as a health signal. σ_6 — the sixth, padded so that a matrix with too few rows reads exactly zero — is the project's spectral health metric.

Padding matters. An earlier version used "the smallest singular value of whatever shape the matrix happens to be". With two markers, L is 4×6 , its smallest singular value is a healthy-looking **9.97e-2**, and the true sixth one is **exactly 0** because two whole directions are invisible. The metric could not see a null space; padding to a fixed length fixes it.

The ambiguity that shows up everywhere here

Look at two columns of the interaction matrix (chapter 7): the one for sliding sideways (v_x) contains $-1/Z$, and the one for rotating about the vertical axis (ω_y) contains $-(1 + x^2)$. For a small target that is far away, x is small and both columns are nearly constant — so **sliding right and rotating right look almost the same in the image**.

"Almost the same" means: the *difference* between them is a direction with a very small singular value. In this project's benchmark that pair sits at **0.8% of the strongest** direction, an amplification of about **130x**, and it corresponds to the camera *orbiting* the target: $v_x = -Z \cdot \omega_y$. Chapter 25 is what happens when a controller's command lands there.

Say it like this

An SVD says a matrix rotates, stretches along six axes, and rotates again. The stretches are the singular values, and inverting the matrix inverts them, so a direction the image can barely see gets amplified by one over a very small number. Every lurch in this project is that amplification, and the two responses are to truncate those directions or to watch them and change control law.

5. Feedback control in one chapter

The loop

Feedback control is three steps, repeated forever:

1. **Measure** where you are.
2. **Compare** with where you want to be. The difference is the **error**.
3. **Act** in a way that reduces the error.

A thermostat does it with temperature. A servo does it with position. This project does it with the positions of markers in a picture.

The step that matters is the third one: *how much* do you act? Act too weakly and it takes forever. Act too strongly and you overshoot, come back, overshoot again — the system oscillates, or goes unstable and never settles.

Proportional control and exponential decay

The simplest choice is to act in proportion to the error:

$$\text{action} = -\lambda \cdot \text{error}$$

λ (lambda) is the **gain**. The minus sign says "move against the error".

Ask what this does over time. If the action *is* the rate of change of the error, then

$$\dot{e} = -\lambda e$$

and that equation has one solution:

$$e(t) = e(0) \cdot \exp(-\lambda t)$$

The error decays exponentially, never quite reaching zero but halving every $\ln 2 / \lambda$ seconds. This is the target behaviour of every IBVS controller here, and $\lambda = 0.5$ unless a chapter says otherwise.

Two honest observations about exponential decay:

- It is **fast when the error is large**. Commanded speed is proportional to error, so at the start of a move the controller asks for the most. That is exactly when a wrong model hurts most, and it is why "what is the peak commanded velocity" is the measure used for the spike failures.
- It **never overshoots** in a simple system. When these controllers do lurch, it is never because the gain is too high; it is because the map between action and error — the interaction matrix — is wrong or nearly singular.

Doing it in discrete steps

A computer cannot act continuously. It samples every Δt , computes, and holds the command until the next sample. The study uses $\Delta t = 0.033 \text{ s}$ (30 Hz); the Gazebo arm runs at $\Delta t = 0.1 \text{ s}$ (10 Hz), which is a realistic rate for a vision loop with a detector in it.

Sampling has a price: for Δt seconds the robot acts on stale information. A loop that is stable in continuous time can oscillate when sampled too slowly, and any delay — camera exposure, detection time, the controller's own computation — adds to it. The arm chapters keep everything on **simulation time** for exactly this reason: when recording slowed the simulator to a quarter of real time, an earlier version used a wall-clock timer and silently changed the control rate.

Why a decision, not a gain, is the subject here

The classic knobs of control — gain, filtering, feed-forward — are not what this project tunes, and it is worth being able to say why.

The failures in part 2 are not "the gain is wrong". They are cases where the relationship the controller is inverting has stopped being trustworthy: a direction has gone blind, or a substitute measurement has stopped meaning what it meant. No value of λ fixes that. Turning the gain down makes the same wrong motion happen more slowly.

What does fix it is choosing a **different control law** for the duration, and that is a decision, made every step, on evidence. The evidence is a property of the geometry, never of the error — because chapter 10 shows the error reads perfect during the most spectacular failure in the book.

Say it like this

Feedback is measure, compare, act. Acting proportionally to the error gives exponential decay, which is the behaviour every controller here asks for. The failures in this project aren't gain problems — they're cases where the map from action to error has gone wrong, and no gain fixes that. So the variable I tune is not a gain, it's which control law runs this step.

Part 1

Visual servoing, derived

6. The servo loop

In plain words

Guiding your hand to a doorknob: you don't measure centimetres, you look, see the knob is too far left, move right, look again. Repeat until it's where it should be.

That is the whole loop. The robot's version replaces "where the knob is in your view" with "where six marker dots are in the camera image", and replaces "move right" with six numbers describing how the camera should move.

Two ways to build it

Position-based (PBVS). Use the image to estimate the full 3D pose of the part, then drive the *pose* error to zero.

- Good: you reason in metres and degrees, which is intuitive, and the path through space is a straight line.
- Bad: you need a model of the target and a well-calibrated camera, and while the pose error looks fine the markers can drift out of the picture, at which point you have nothing.

Image-based (IBVS). Define the error *in the image* and drive that to zero. No pose estimate anywhere.

- Good: features stay in the frame, because keeping them where they should be in the picture *is* the objective. Robust to calibration error.
- Bad: the relationship between camera motion and image motion is nonlinear and depends on depth, and the path through space can be strange.

This project is entirely IBVS, because the interesting failure modes live in that nonlinear relationship.

The pieces

Features s . What the camera measures. Here: six marker centres, each a pair of normalised coordinates (chapter 3), so s is 12 numbers.

Desired features s^* . Where those measurements should be when the robot is in the right place. Recorded once by putting the robot at the goal and looking, or computed from the known target geometry and the desired standoff.

Error $e = s - s^*$. 12 numbers, in image units, not metres.

Command v . Six numbers: the camera's twist (chapter 2).

The loop, per step:

```

1. take a picture
2. find the markers          -> s
3.  $e = s - s^*$ 
4. build the interaction matrix L from s and depth
5.  $v = -\lambda * \text{pinv}(L) @ e$ 
6. send v to the robot, wait dt, repeat

```

Steps 4 and 5 are the whole subject of the next two chapters. Step 2 is perception, which is assumed perfect for most of the study and then made real twice: with an ArUco detector on rendered images (chapter 20) and a blob detector on a simulated arm (chapter 24).

One subtlety: correspondence

Step 3 subtracts two lists. That only means something if entry k of s and entry k of s^* are the **same physical marker**. Getting that pairing right is called **correspondence**, and with six identical dots it is not free.

Ordering the detected dots by angle around their centre fixes their order but not where the order *starts*, so a rotated target can pair every dot with the wrong partner. In the Gazebo cell that produced a beautiful bug: one arm converged to an error of 0.090 while its identical twin diverged to 0.530, running the same code. The fix is to try all six cyclic shifts and keep the one with the smallest residual (chapter 24).

Markers with ids — ArUco, AprilTag — solve this by construction, which is why the shipped ROS node orders detections by id and only falls back to angular ordering when ids are absent.

What "converged" means

Two different measures are used in this book, and they are not interchangeable:

- **Convergence**: the final image error falls below $1e-4$. Used for failures where the question is "does it get there at all".
- **Velocity spike**: the peak commanded speed during the disruption, divided by that same controller's speed just before it. Used for failures where the controller does arrive but lurches on the way.

A single score would hide exactly the thing being measured, which is why the results table in chapter 16 says in bold that its cells compare down a column and never across one.

Say it like this

Image-based visual servoing defines the error directly in the image: where the markers are versus where they should be, in normalised camera coordinates. You never estimate a 3D pose. The loop is: detect, subtract, build the matrix that maps camera motion to image motion, invert it, command the motion. The correspondence step — which detected dot is which — is the part that looks trivial and isn't.

7. Deriving the interaction matrix

This is the one derivation to know by heart. Everything in parts 2 and 3 is a consequence of the shape of this matrix.

The question

If the camera moves with twist $\mathbf{v} = (v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)$, how fast does a marker move in the image?

Answer it and you have the rule linking action to observation, which is exactly what a controller needs to invert.

Step 1: how the point moves in the camera's frame

A marker is fixed in the world; the camera moves. So in the **camera's** frame the point appears to move backwards. From chapter 2:

$$\dot{\mathbf{p}} = -\mathbf{v}_{lin} - \boldsymbol{\omega} \times \mathbf{p} \quad \text{with } \mathbf{p} = (X, Y, Z)$$

Writing the cross product out:

$$\begin{aligned} \dot{X} &= -v_x - (\omega_y \cdot Z - \omega_z \cdot Y) \\ \dot{Y} &= -v_y - (\omega_z \cdot X - \omega_x \cdot Z) \\ \dot{Z} &= -v_z - (\omega_x \cdot Y - \omega_y \cdot X) \end{aligned}$$

Step 2: how the image coordinate depends on the point

From chapter 3, the projection is $x = X/Z$. Differentiate it with the quotient rule:

$$\dot{x} = (\dot{X} \cdot Z - X \cdot \dot{Z}) / Z^2 = \dot{X}/Z - (X/Z) \cdot (\dot{Z}/Z) = \dot{X}/Z - x \cdot \dot{Z}/Z$$

The same for $y = Y/Z$:

$$\dot{y} = \dot{Y}/Z - y \cdot \dot{Z}/Z$$

Step 3: substitute and collect

Take the $\dot{x}$ equation and put Step 1 into it.

$$\begin{aligned} \dot{X}/Z &= (-v_x - \omega_y \cdot Z + \omega_z \cdot Y) / Z && \text{(using } Y/Z = y) \\ &= -v_x/Z - \omega_y + \omega_z \cdot y \\ x \cdot \dot{Z}/Z &= x \cdot (-v_z - \omega_x \cdot Y + \omega_y \cdot X) / Z && \text{(using } X/Z = x) \\ &= -x \cdot v_z/Z - x \cdot y \cdot \omega_x + x^2 \cdot \omega_y \end{aligned}$$

Subtract the second from the first:

$$\dot{x} = -v_x/Z + x \cdot v_z/Z + x \cdot y \cdot \omega_x - (1 + x^2) \cdot \omega_y + y \cdot \omega_z$$

And by the identical route for y :

$$\dot{y} = -v_y/Z + y \cdot v_z/Z + (1 + y^2) \cdot \omega_x - x \cdot y \cdot \omega_y - x \cdot \omega_z$$

Step 4: write it as a matrix

Those two lines are linear in the six components of $\mathbf{v}$, so pull the coefficients out into rows:

$$\begin{aligned}\dot{x} &= [-1/Z, \quad 0, \quad x/Z, \quad x \cdot y, \quad -(1 + x^2), \quad y] \cdot \mathbf{v} \\ \dot{y} &= [\quad 0, \quad -1/Z, \quad y/Z, \quad 1 + y^2, \quad -x \cdot y, \quad -x] \cdot \mathbf{v}\end{aligned}$$

Stack one such pair per marker and you have the **interaction matrix** $\mathbf{L}$, shape $2N \times 6$, with

$$\dot{\mathbf{s}} = \mathbf{L} \cdot \mathbf{v}$$

That is `interaction_matrix()` in `ibvs_core.py`, and every entry is checked by finite difference in `fd_check.py` — nudge the camera, measure where the dots actually went, compare. A derivation you have checked against the simulator is worth more than one you have checked against a paper.

Reading the matrix

Four things are visible by inspection, and all four become chapters later.

Depth appears only in the translation columns. $1/Z$ sits under v_x , v_y , v_z and nowhere else. So rotation can be controlled without knowing depth; translation cannot. Every "how far away am I" problem in this book traces back to this.

Translation effects shrink with distance, rotation effects do not. At $Z = 2$ m the v_x column is half what it is at 1 m, while $-(1 + x^2)$ is unchanged. Far away, the camera can rotate cheaply and must translate a long way to achieve anything.

Sliding and rotating look alike. Compare the v_x column, $-1/Z$, with the ω_y column, $-(1 + x^2)$. For a small target near the image centre, x is small, so one is roughly $-1/Z$ and the other roughly -1 . They differ only by a scale — which means a *combination* of them produces almost no image motion at all. That combination is the near-null direction of chapters 4 and 25, and it works out to $v_x = -Z \cdot \omega_y$: the camera **orbiting** the target.

A flat target seen face-on is the worst case. Every marker then has the same Z , so the whole v_z column is proportional to the position x , which is also what a rotation does. Depth motion and rotation become hard to distinguish, the sixth singular value gets small, and — counter-intuitively — **tilting the target improves conditioning**, because it spreads the depths out. That single fact is why a signal computed from the matrix and a signal computed from the picture disagree about a tilted part (chapter 25).

Where z comes from

L needs a depth per marker, and a single camera does not measure it. Implemented options (`depth_mode` in `run_ibvs`):

- `true` — exact depth, simulation only, used as the control condition.
- `desired` — the depth at the goal pose. Standard practice.
- `constant` — one number for every marker. What the Gazebo arm uses (0.26 m), which is what a real cell with no depth sensor would do.

Errors in z scale the translation columns. The controller still converges because the direction of the correction stays roughly right; the speed is off. It is a remarkably forgiving dependency, which is one of the reasons IBVS is popular.

Say it like this

You differentiate the projection $x = X/Z$, substitute the rigid-body velocity of a point seen from a moving camera, and collect terms. Out comes two rows per feature, six columns, mapping camera twist to image velocity. Reading those rows tells you everything that goes wrong later: depth only enters the translation columns, and the sideways-translation column and the rotation-about-vertical column are nearly parallel, which is the weak direction that later swings the camera edge-on.

8. The control law, and turning it into motion

Deriving the law

We want the image error to die away exponentially (chapter 5):

$$\dot{e} = -\lambda e$$

We know how the image moves when the camera moves (chapter 7):

$$\dot{s} = L v \quad \text{and since } s^* \text{ is fixed,} \quad \dot{e} = \dot{s} = L v$$

Set them equal and solve for the command:

$$L v = -\lambda e \quad \rightarrow \quad v = -\lambda \cdot L^+ e$$

That is classic IBVS in one line, and it is `run_ibvs()` in `ibvs_core.py`.

What the pseudo-inverse is doing here

With six markers, L is 12×6 : twelve equations, six unknowns. No v makes every equation exactly true, so L^+ returns the **least-squares** answer, the motion that comes closest to the requested image change.

With two markers, L is 4×6 : four equations, six unknowns. Now many motions fit exactly, and L^+ returns the **minimum-norm** one — the smallest.

Remember that difference. In chapter 12 it is the entire reason classic IBVS survives with two markers while the 2001 partition explodes: one is protected by the minimum-norm rule, and the other has been rearranged into an exactly determined 4×4 with no slack left.

Why this is not the same as "move toward the target"

A natural but wrong mental model is that the controller looks at the dots, notices they are up and to the left, and moves up and to the left. It does something subtler: it asks for the *fastest reduction of the image error in a least-squares sense*, and any camera motion that achieves that is acceptable to it.

If backing away happens to make all twelve numbers smaller more efficiently than rotating does, the controller backs away. That is not a bug in the implementation; it is the exact solution to the problem as posed. Chapter 10 is what that looks like when it goes wrong: the camera flies 68 metres backwards while the error decays beautifully the whole time.

From twist to motion: integrating

The command is a velocity; the simulation needs a pose. Apply it for one step with the matrix exponential (chapter 2):

```
cTo ← se3_exp(-v · dt) · cTo
```

On a real robot this step is not free, and chapter 24 does it properly: the camera twist is mapped through the arm's geometric Jacobian into joint velocities, clipped to the joints' limits, integrated for one cycle, and sent to a joint trajectory controller.

The loop, in code

```
for k in range(steps):
    P_c = transform_points(cTo, P_o)      # markers in the camera frame
    Z   = P_c[:, 2]
    s   = project(P_c)                    # x = X/Z, y = Y/Z
    e   = (s - s_star).reshape(-1)
    L   = interaction_matrix(s, Z)
    v   = -lam * pinv(L) @ e
    cTo = se3_exp(-v * dt) @ cTo
```

Ten lines, and every failure in part 2 happens inside them. The variations that follow — truncation, the partition, the switch — change exactly one of these lines each.

Tuning knobs, and which ones matter

- **λ (gain).** Sets the speed. $\lambda = 0.5$ throughout the study.
- **dt.** 0.033 s in the study, 0.1 s on the arm.
- **depth mode.** Affects the scale of the translation columns.
- **τ (truncation).** Introduced in chapter 15, the one constant with a documented failure case of its own.

What is *not* a knob: the geometry. When the markers stop carrying the information the controller needs, no setting of λ , dt or depth recovers it. That is what makes the failures in part 2 interesting rather than a tuning exercise.

Say it like this

Ask for exponential decay of the image error, substitute the interaction matrix, and solve: v equals minus gain times the pseudo-inverse of L times the error. The pseudo-inverse is least-squares when you have plenty of features and minimum-norm when you don't, and that distinction decides who survives the two-feature case. Integrating the twist onto the pose uses the matrix exponential, and on the real arm it goes through the robot's Jacobian instead.

9. The harness: what is actually run

A result is only as good as the setup that produced it, so this chapter states the setup completely: targets, scenarios, measures, seeds and files.

The two targets

- **A six-point ring.** Six markers on a circle of radius 0.05 m, on a plane. Viewed from about 0.6 m. The main target.
- **A four-point square.** Half-width 0.1 m, viewed from 0.8 m. Used for the camera-retreat scenario.

Two targets, not one, because a threshold calibrated on one does not transfer to the other: the healthy 1st percentile is **3.87e-3** for the ring and **1.74e-2** for the square, 4.5× apart (chapter 17).

The four scenarios

scenario	what happens	why
camera retreat	the goal is the same pose rotated 180° about the optical axis	the famous coupling failure; positive control
feature dropout	3 of 6 markers vanish for steps 30–90	conditioning failure
two features	4 of 6 vanish for steps 30–90	tests the partition's reduced solve
collapsed target	all 6 visible, squashed toward a line in 3D (scale 0.02)	information failure that counting cannot see

Occlusions are a function of the step number, so every controller sees exactly the same disruption at exactly the same time. That is what makes the comparison fair, and it is the same principle as putting both Gazebo cells in one physics world (chapter 23).

The measures, and why they differ

Convergence — final image error below $1e-4$ — for retreat and collapse, where the question is whether the controller arrives at all.

Velocity spike — peak commanded speed during steps 30–90, divided by that controller's own speed at step 29 — for dropout and two-features, where every controller eventually arrives and the question is how violently.

Dividing by the controller's *own* pre-disruption speed matters: it asks "did this controller lurch relative to how it was already moving", which is comparable across laws that run at different speeds. A spike below 1 means the controller slowed down when information was lost, which is the desired behaviour.

Clean cost — steps to drive the error below 1% of its initial value on an undisturbed run, relative to classic IBVS. The threshold is part of the definition: the same comparison gives +2.4% at 50% and +7.4% at 0.01%, so quoting "+13%" without saying "at 1%" is meaningless.

Pose error — distance between the final camera position and the true goal position — used in the observability chapter, where the image error can be tiny while the robot is in the wrong place.

Reproducibility rules

- Thresholds are the healthy 1st percentile over **4000 random poses, seed 0**.
- Every scenario is deterministic given its seed; noise studies state theirs.
- Every number quoted in the README is produced by a script in the repo, and the ones that matter are locked by a test so a later change cannot move them quietly.

The files

file	what it is
<code>ibvs_core.py</code>	SE(3) helpers, camera, projection, interaction matrix, classic IBVS, σ_6 , scenarios
<code>partitioned.py</code>	the 2001 partitioned law
<code>truncated.py</code>	adaptive-rank pseudo-inverse and the truncation-only controller
<code>switched.py</code>	the runtime switch, the six rules, and healthy-pose calibration
<code>tests.py</code>	177 regression checks
<code>fd_check.py</code>	16 finite-difference checks
<code>compare.py</code>	regenerates the results table
<code>tau_sweep.py</code> , <code>noise_study.py</code> , <code>predict.py</code>	the sensitivity, noise and forward-prediction studies
<code>mj_*.py</code>	the rendered-camera port with a real ArUco detector
<code>blindspot/</code>	the shipped library: guard, units, calibration
<code>ros2_ws/src/blindspot_ros/</code>	the ROS 2 guard node and the RViz simulation
<code>ros2_ws/src/blindspot_arm/</code>	the Gazebo UR5e cells and the control law they run

How to run it

```
python3 -m venv .venv && .venv/bin/python -m pip install -r requirements.txt
.venv/bin/python tests.py          # must print 177/177 and 16/16
.venv/bin/python compare.py       # regenerates the results table
```

The dependency list is numpy, scipy and matplotlib. The rendered-camera scripts additionally need `mujoco` and `opencv-contrib-python`; the ROS parts need ROS 2; this

book's PDF needs `reportlab`. None of those are required to reproduce the control results, which is deliberate: the core study should be runnable by anyone in one minute.

Say it like this

Two targets, four scenarios, and a measure chosen per failure rather than one score for everything — convergence where the question is whether it arrives, peak commanded velocity relative to the controller's own pre-disruption speed where the question is whether it lurches. Occlusions are scripted by step number so every controller meets the identical disruption, thresholds come from a fixed seed, and the whole core study runs on numpy alone.

Part 2

Four ways to go blind

10. Failure 1: camera retreat

In plain words

Ask the controller to do something simple: keep the camera where it is, but rotate it half a turn about the direction it is looking. Like turning a photo upside down without moving.

It flies the camera **68 metres backwards** and never comes back.

Nothing is broken. No matrix is singular, no marker is missing, the detector is perfect. The controller is solving exactly the problem it was given, and the answer to that problem is insane.

Why it happens

Recall what the law actually optimises (chapter 8): the motion that reduces the twelve image-error numbers fastest, in a least-squares sense. It does not care how it gets there.

Now picture the markers as a hexagon in the image that has to end up rotated by 180° . Two motions reduce the error:

1. **Rotate** about the optical axis. Every marker travels along a circular arc — a long way around — before anything lines up.
2. **Back away**. Everything shrinks toward the image centre, which reduces all twelve numbers immediately, and it costs nothing in the first instant.

Locally, option 2 is the better bargain, so the least-squares solution takes it. And it keeps taking it, because the trade never reverses: the further back you go, the smaller the errors, and the rotation still has not happened. The camera commits to a spiral it never finishes.

This is a **coupling** failure. It is not a numerical one. Nothing about the matrix is ill-conditioned at any point on that trajectory, so no amount of truncation, regularisation or careful inversion helps — measured: adaptive-rank truncation still diverges, at 13.77 m.

The measurement that changed the project

Throughout the retreat, compute the alignment between the commanded motion and the error — the cosine of the angle between them, which says "is the controller pushing in the right direction?"

It is exactly **−1.0000** at every single step, while the camera flies 68 m away.

By its own arithmetic, the controller is perfect. The error is decaying smoothly. Every quantity a naive monitor would watch says everything is fine.

So image error is a useless health signal. Anything that watches the error reports perfect health during the most spectacular failure in the book. That finding decided the design of everything afterwards: every candidate health signal in this project is a property of the *geometry the controller depends on*, never of its error.

Why it is in the project at all

Retreat was solved before this project started — by 2½D visual servoing (Malis, Chaumette & Boudet, 1999) and by the partitioned scheme (Corke & Hutchinson, 2001), which is the one studied here.

So it is not a contribution. It is a **positive control**: a failure with known ground truth. If a proposed health signal does not fire on retreat, the signal is useless; if it fires there and nowhere sensible, it is a smoke alarm that only rings at parties. Having one failure with a known answer is what makes the other three interpretable.

The numbers

controller	furthest distance	outcome
classic IBVS	68.19 m	diverges
adaptive-rank truncation	13.77 m	diverges
partitioned (2001)	0.80 m	converges
combined, and every switched rule	0.80 m	converges

0.80 m is the starting distance, so "0.80 m" means the camera never backed up at all.

The catch, discovered much later

That benchmark starts from an **exactly** symmetric pose: a pure 180° rotation about the optical axis with no offset. In that configuration the partitioned law's sideways command is zero by symmetry — measured 2.1e-15, numerically zero.

Start it 2 cm or 3° off, and the symmetry breaks. What happens then is chapter 25, and it is the most interesting result in the second half of this book: the camera swings nearly edge-on, one guard catches it and pays for it with a 2–3 m retreat, and the other stays silent and loses the target entirely in 3 runs out of 100.

A benchmark that starts from a perfectly symmetric pose is not lying, but it is answering an easier question than the one you think you asked.

Say it like this

The classic failure: ask for a 180° rotation about the optical axis and the camera flies 68 metres backwards, because retreating reduces the image error faster than rotating does and the least-squares solution takes that bargain. It's a coupling problem, not a conditioning one, so truncation doesn't help. The part I took from it is that the controller's own error signal reads minus one — perfect alignment — the entire time, so error is useless as a health measure.

11. Failure 2: feature dropout

In plain words

Half the markers disappear for a couple of seconds — an arm passes in front, a fixture blocks them, the lighting changes. Three of six are gone, then they come back.

The controller does not pause or slow down. It commands a motion **28.7 times** larger than it was moving before.

Why it happens

Six markers give twelve equations for six unknowns: comfortably redundant. Three markers give six equations for six unknowns: exactly enough, in principle, but with no slack.

More importantly, the three that remain are on one side of the target, so the *shape* they make is thinner. In the language of chapter 4, one of the six singular values of the interaction matrix has collapsed toward zero: there is now a way the camera can move that these three markers can barely see.

And the controller inverts that matrix. Any component of the requested image change that points along the weak direction gets multiplied by $1/\sigma$, which is now a big number. The command jumps.

This is the textbook **conditioning** failure, and unlike retreat it is a numerical problem with a numerical answer.

The fix that works

Truncation (chapter 15): look at the singular values, and refuse to move along any direction weaker than a fraction τ of the strongest.

controller	spike
classic IBVS	28.7×
adaptive-rank truncation	0.9×
partitioned (2001)	1.1×
switched, area guard	0.5×

A spike **below 1** means the controller moved more slowly during the disruption than before it. That is the behaviour you want: information went missing, so do less. 28.7× is the opposite — information went missing, so sprint.

What this case teaches about thresholds

This scenario is also where a subtle calibration problem shows up, and it is worth understanding because it looks like a healthy result until you know why.

If the guard's threshold is calibrated with all six markers visible, then during dropout — three visible — the statistic is compared against a threshold built for a different situation. The margin comes out at **1.19x**: technically above the line, but only just, and for a structural reason rather than a physical one. A subset of points always has a smaller polygon than the full set, so the comparison is unfair by construction (the mathematical name is singular-value **interlacing**).

Calibrating **per visible count** — a separate threshold for 6, 5, 4, 3 markers — restores the same operating point to **45.9x** of its threshold. Same run, same physics, honest comparison. Chapter 17 has the procedure.

That fix is in the shipped library, and the README labels it plainly as "a fix for a defect, not a free property".

A note on what "the same disruption" means

Every controller in the table above meets the identical occlusion: the same three markers, hidden from step 30 to step 90, scripted by step number rather than by time or by pose. Without that, a slower controller would meet the occlusion at a different place on its trajectory and the comparison would be meaningless.

The same principle drives the two-arm Gazebo demo later: both cells live in **one** physics world so they see the same part fold at the same instant (chapter 23).

Say it like this

Occlude three of six markers and classic IBVS spikes 28.7x — that's the textbook conditioning failure, where one direction of the interaction matrix goes blind and the inverse amplifies it. Truncation fixes it properly, down to 0.9x. This case is also where I found that a guard threshold calibrated at six markers is structurally unfair at three: the margin reads 1.19x, and conditioning the calibration on the visible count makes the same instant read 45.9x.

12. Failure 3: two features left

In plain words

Now hide four of the six markers. Two left.

Classic IBVS shrugs: spike **0.9×**, it moves *more gently* than before. The published 2001 fix — the good one, the one that solves camera retreat — commands a motion **288.4 times** larger than it was moving.

Same disruption, same instant, same target. The difference is entirely in the structure of the two control laws.

Why the plain law survives

Two markers give four numbers, so four equations. The camera has six degrees of freedom. Four equations, six unknowns: **under-determined**. Infinitely many motions satisfy the request exactly.

The pseudo-inverse has to pick one, and it picks the **smallest** (chapter 4). So when information disappears, classic IBVS is handed the gentlest motion consistent with what it can still see.

It is not clever. It is not intentional. It is a property of the minimum-norm solution, and it protects the controller by accident.

Why the 2001 fix does not

The partition (chapter 14) improves retreat by taking two degrees of freedom out of the inverse and driving them from direct image measurements. The other four still come from a reduced matrix, L_{xy} .

Count again with two markers: L_{xy} is **4×4**. Four equations, four unknowns, exactly determined.

There is now no redundancy to average over and no minimum-norm rule to fall back on, because there is exactly one solution and the law must take it. If that 4×4 is ill-conditioned — and with two markers it is — the single solution is enormous.

The published fix for failure 1 is the cause of failure 3. That sentence is the hinge of the whole project.

The numbers, and the gap that defines the work

controller	spike
classic IBVS	0.9×
adaptive-rank truncation	0.9×
partitioned (2001)	288.4×
both combined (fixed)	11.6×

controller	spike
switched, any guard rule	0.5x

Adding truncation on top of the partition — the "combined" controller — reduces 288x to 11.6x. Better, and still a lurch: eleven times the previous speed, at the moment the robot knows least.

11.6x is the gap. The founding hypothesis was that closing it needed a learned policy. The bar for abandoning that plan was set in advance at 2x. A one-line rule reached **0.5x**.

Why truncation alone cannot rescue the partition

Worth being precise, because it is a natural objection. Truncation works by declining to move along directions that have gone weak. But the partition has already *committed* two degrees of freedom to a separate path — they are computed from the area and the line angle, not from the inverse — so the four that remain must be solved exactly as a block. Truncating inside a 4x4 that has to produce one specific answer either does nothing or removes a direction the law still needs.

The only real fix is to stop using the partition while the situation lasts, which is the switch (chapter 16).

The minimum-feature honesty note

To let a genuine two-feature step happen at all, the simulation has to be told that two is acceptable (`min_features=2`); by default the harness falls back to using every marker still available when fewer than three are visible, which is what a competent implementation would do. Studying the two-feature regime means deliberately switching that protection off, and saying so.

Say it like this

With two markers, classic IBVS is under-determined, so the pseudo-inverse hands it the minimum-norm solution and it moves gently: 0.9x. The 2001 partition has taken two degrees of freedom out of the inverse, so its reduced solve is an exactly-determined 4x4 with no slack, and it spikes 288x. Adding truncation gets it to 11.6x, and that 11.6 is the number the rest of the project exists to close. A one-line runtime rule gets it to 0.5.

13. Failure 4: the collapsed target

In plain words

This is the one that should worry you.

Nothing is hidden. All six markers are visible, crisply detected, right where the detector expects them. Every safety check based on "can I still see my markers?" says yes, continuously, the whole time.

And the controller cannot finish the job, because the six markers have stopped carrying the information it needs. Their geometry has flattened toward a line.

Why a flat arrangement carries less information

Six points in a ring pin the camera down in every direction: move left and the ring shifts, roll and it spins, approach and it grows.

Squash that ring toward a line and one of those is lost. Rotating about the line barely changes the picture, because points *on* an axis of rotation do not move when you rotate about it. The camera can move a long way with almost no evidence appearing in the image — which is exactly the definition of a weak direction from chapter 4.

In the benchmark, the four singular values of the matrix the partition must invert come out as

```
L_xy = [ 3.68, 3.66, 0.0124, 0.0037 ]
```

Three orders of magnitude between the strongest and the weakest. Inverting that multiplies anything landing in the weak direction by about 270.

Why counting features cannot see it

The feature count is **6** at every step. A rule that watches the count reports perfect health, permanently, while the controller fails.

This is the single strongest argument for the shipped guard, and it is why the guard watches a *geometric* quantity rather than a count. It is also the case the ROS demo shows explicitly: six markers present, guard correctly dropping the partition (chapter 22).

What each controller does

controller	final image error	verdict
classic IBVS	8e-11	converges
adaptive-rank truncation	2.4e-5	converges
partitioned (2001)	7.3e-3	floors, never reaches 1e-4
both combined (fixed)	2.0e-3	floors

controller	final image error	verdict
switched, feature count	2.0e-3	floors — counting cannot see it
switched, σ_6 or area guard	2.4e-5	converges

Truncation converges because it is allowed to decline the blind direction. The partition cannot, because it has already committed those degrees of freedom to its reduced solve, so the blind direction gets inverted anyway.

Two corrections I had to make here

Both were mine, both were wrong first, and both are in the repository next to the result rather than quietly fixed.

The mechanism is not "the area goes to zero". The obvious story — the polygon area shrinks, so the depth term $\log(\sigma^*/\sigma)$ blows up — is false. The *desired* area shrinks along with the current one, the ratio stays at **1.4980**, and the commanded v_z is unchanged at every collapse level. The failure is in the reduced inverse, not the depth feature. I checked because the story was too tidy.

This case must be judged on convergence, not on its velocity spike. At the shipped $\tau = 1e-3$, the truncation threshold happens to land at **98.78%** of the smallest singular value — a 1.2% margin. So whether a direction is kept or dropped flips on a coin, and the spike moves from **5.96** ($\tau=1e-4$) to **0.33** ($\tau=1e-2$), where it matches truncation-only's 0.32. A number that moves by 18× when you nudge a constant is not evidence.

The convergence half of the row is τ -independent — the combined controller floors above $1e-4$ at every τ from $1e-4$ to $1e-1$ — so that is the half that is reported. This is dead claim 5 in chapter 19.

How it is produced in the study, and a limit that follows

The collapse is applied directly to the target geometry: the ring's coordinates are squashed along one axis by a factor c , with $c = 0.02$ for the benchmark.

That is easy in simulation and awkward in reality, which produces one of the most interesting limits in the project: **with fiducial markers this case is structurally untestable**. The degeneracy is the absence of 2D extent, and an ArUco marker needs 2D extent to decode. Reproducing the verdict needs $c \leq 0.05$; the smallest collapse that still leaves six decodable markers at this working distance is $c = 0.35$ — a factor of seven, and not a resolution limit, because it persists at 2560 px wide.

So in that regime the guard's trigger condition and the detector's failure coincide exactly. Real deployments need corner or edge features there, not markers. That is why the Gazebo cell later uses plain dark dots, which stay detectable while the geometry collapses (chapter 26).

Say it like this

The collapsed target is the case that matters: all six markers visible and perfectly detected, and the geometry has gone flat, so one camera motion is nearly invisible in the image. Feature counting says six out of six the entire time while the partitioned controller floors short of the goal. That is the argument for guarding geometry rather than counting features — and it's also where I had to retract two of my own explanations, including a velocity spike that turned out to be an artifact of the truncation constant.

Part 3

Fixes, and what they cost

14. The 2001 partition

In plain words

Camera retreat happens because the solver gets to choose *how* to reduce the image error, and it chooses badly. Corke and Hutchinson's fix in 2001: take that choice away for the two motions that cause the trouble, and measure them straight off the picture instead.

- **How far away am I?** Look at the size of the patch the markers cover.
- **How twisted am I?** Look at the angle of a line between two chosen markers.

The other four motions are still solved by the maths, but now with four unknowns instead of six. Retreat disappears, because backing away is no longer something the solver can invent — the distance term is measured.

The two measured degrees of freedom

```
sigma = sqrt(polygon area of the markers in the image)
alpha = angle of the line from marker 0 to marker 1

vz = lam_z * log(sigma_star / sigma)
wz = lam_a * wrap(alpha - alpha_star)
```

Why the square root, and why the logarithm

This is the elegant part of the 2001 scheme and it is worth deriving.

A planar target seen face-on at depth Z projects to a polygon whose area scales as $1/Z^2$ (everything divides by depth, chapter 3, and area is two dimensions of it). Take the square root and you get a quantity that scales as $1/Z$:

```
sigma ∝ 1/Z          so          sigma_star / sigma = Z / Z_star
```

The ratio of the measured feature to its desired value **is** the ratio of the depths — with no need to know either one. Take the logarithm and you have a depth error in log space:

```
log(sigma_star / sigma) = log Z - log Z_star
```

Driving v_z proportionally to that is a proportional controller on the *logarithm* of depth. Two consequences: it is scale-free, so a 2× error far away and a 2× error close up produce corrections of the same size in log terms; and it approaches smoothly without overshoot, because the error shrinks as the camera closes in.

And the entire thing is computed from the picture. No depth sensor, no target model, no calibration of distance. That is why the scheme is attractive, and **it is also the vulnerability this whole book is about**: σ stands in for depth only as long as nothing else changes the patch size. Tilt the part (chapter 25) or fold it (chapter 26) and the substitute lies.

Why the angle, and the sign

Rolling the camera by ω_z rotates the whole image by exactly the opposite amount, so $d(\alpha)/d(\omega_z) = -1$ exactly. That makes the correct sign in the law positive. It is one line of code and it is the kind of thing that is 50/50 if you reason it out, so it was checked by finite difference instead — nudge ω_z , watch the measured angle, compare. Getting it backwards makes the camera rotate away from the goal, which looks like a control problem and is not.

The pair of markers used for α must be **fixed in advance**, and the same pair in the current and desired image. Choosing it from the current picture — say the longest visible line — makes α jump every time the choice changes, which on a symmetric target happens constantly.

The remaining four degrees of freedom

Split the interaction matrix by columns: L_z for v_z , ω_z and L_{xy} for v_x , v_y , ω_x , ω_y . Then

$$\dot{s} = L_{xy} \cdot v_{xy} + L_z \cdot v_z$$

We want $\dot{s} = -\lambda e$, and v_z is already decided, so

$$v_{xy} = -\text{pinv}(L_{xy}) \cdot (\lambda e + L_z v_z)$$

The term $L_z v_z$ is the **coupling term**, and it is **added**. The reason is physical: v_z and ω_z are about to move every marker in the image, so the other four degrees of freedom must cancel that motion *in addition to* driving the error down. Subtracting it instead makes the controller fight itself, and that was one of the two sign bugs finite differences caught.

Why retreat dies

Retreat requires the solver to choose backing away. In this law, the camera's depth motion comes from one measurement — is the patch smaller than it should be? — and nothing else can ask for it. So the pathology has nowhere to live.

controller	retreat
classic IBVS	68.19 m, diverges
partitioned (2001)	0.80 m, converges

0.80 m is where it started. It never backed up.

What it costs

Two features left: the reduced solve becomes an exactly determined 4×4 with no slack, and the spike is **288.4×** (chapter 12).

Collapsed target: four degrees of freedom are forced through an ill-conditioned L_{xy} that truncation is no longer free to prune, and the run floors at **7.3e-3** (chapter 13).

Clean cost: on an undisturbed run, **+13%** more steps to drive the error below 1% of its initial value. Worth quoting because a fix with no measured downside is usually a fix nobody measured.

The idea that follows

A law that is right in some situations and wrong in others should not be a design-time choice. It should be a runtime one — which is chapter 16, and the only remaining question is what to switch on.

Say it like this

The 2001 partition removes camera retreat by driving two degrees of freedom from direct image measurements instead of from the inverse: the square root of the marker polygon's area, which scales as one over depth so its log is a depth error you never needed a depth sensor for, and the angle of a line between two fixed markers. It genuinely fixes retreat. It also makes the remaining solve exactly determined, which is why it spikes 288x on two features and can't converge on a collapsed target.

15. Truncation: declining to move blind

In plain words

If part of your information has gone missing, the honest response is to not act on that part.

Truncation does exactly that. Before inverting the matrix, look at its six directions and how strongly each one shows up in the image (chapter 4). Any direction that has gone too weak to trust, skip it — the robot simply does not move that way this step. You give up control of something you could not see anyway, and in exchange you are not amplified by one over a tiny number.

How it works

```
U, S, Vt = svd(L)
keep = S > rel_tau * S[0]          # relative to the strongest direction
S_inv = where(keep, 1/S, 0)        # weak directions contribute nothing
L_plus = (Vt.T * S_inv) @ U.T
```

Two details that matter:

The threshold is relative, not absolute. `rel_tau × S[0]` scales with the problem, so the same constant means the same thing at different distances and target sizes. An absolute threshold would need re-tuning for every geometry.

It is adaptive. The rank is recomputed every step from the current image, so the controller uses five directions during an occlusion and six again afterwards, without any mode switch or timer.

What it fixes, and what it does not

failure	truncation
feature dropout	fixes it: 28.7× → 0.9×
two features	fine, 0.9× — but classic IBVS was already fine there
collapsed target	converges , 2.4e-5
camera retreat	no help at all: still diverges, 13.77 m

The pattern is exact and worth stating: truncation cures **conditioning** failures and is powerless against **coupling** failures. Retreat has a perfectly healthy matrix — nothing is weak, so there is nothing to truncate — and the problem is that the solver's preferred solution is a bad path through space.

That is why the project needs both ideas rather than one. Truncation handles the blind-direction failures; the partition handles the coupling failure; and neither handles what the other does.

τ : the one constant, and its documented failure

`rel_tau = 1e-3` is the shipped default, chosen from a sweep in `tau_sweep.py`. It is the only tuning constant in the controller, and the repository states plainly where it fails.

The constructible counter-example: on a collapsed target with an initial rotation about the collapse line, the switched controller at $\tau=1e-3$ **stalls at a pose error of 0.297 m**, while plain classic IBVS **converges exactly** (0.0000 m, image error $1.5e-10$). Truncation is discarding a direction that is weak but still usable, and the guard has handed control to exactly that truncation.

$\tau=1e-5$ completes the same case exactly, so it is a tuning failure rather than a structural one — but the shipped default has a case where intervening is worse than doing nothing, and a single global constant chosen on one geometry is exactly the sort of thing that breaks on someone else's. The README says: if you deploy this, sweep τ on your own geometry rather than inheriting $1e-3$.

A second appearance of the same sensitivity is dead claim 5 (chapter 19): at $\tau=1e-3$ the truncation threshold lands at 98.78% of the smallest singular value on the collapsed-target case, which makes a reported velocity spike an artifact of a 1.2% margin rather than a property of the controller.

Any result that moves a lot when τ moves a little is a result about τ . That is the rule this project took from it.

Say it like this

Truncation filters the pseudo-inverse: run the SVD, drop any direction weaker than a relative threshold, invert the rest. It cures the conditioning failures — the dropout spike goes from $28.7\times$ to $0.9\times$ — and does nothing for camera retreat, because retreat has a healthy matrix and a bad path. The threshold is the only tuning constant in the controller and I documented a case where the shipped value is worse than not intervening at all.

16. The switch: deciding per step

In plain words

Two good control laws, each wrong in a different situation. The obvious move is to stop choosing once, at design time, and choose every step instead — using the partition while it is safe, and the plain truncated law while it is not.

That is the switch. The rest of the project is about what to base that decision on, and how to know the decision is right.

The controller

```
if rule_says_safe(L, L_xy, s_visible):
    v = partitioned_law(...)          # 2001 partition + truncated reduced solve
else:
    v = -lam * pinv_truncated(L, tau) @ e
```

Decided fresh every step, from the current image alone. No timers, no modes, no memory: if the situation improves, the partition comes back the same way it left.

Six candidate rules, not one

rule	what it watches
count	how many markers are visible
rank_margin	numerical rank of L versus the unknowns in L_{xy}
sigma6	the 6th singular value of L , one global threshold
sigma6_n	the same, threshold conditioned on the visible count
area_guard	the polygon area — the partition's own depth feature
alpha_guard	the image distance between the two markers alpha uses

Implementing all six is what makes the eventual claim comparative. "The area guard works" is not a result. "The area guard matched the spectral rule on all four benchmark cases, beat it on one, and here is the case where it is worse" is a result.

Two of these died on measurement and are in chapter 19: `rank_margin` because at two features the reduced matrix is 4×4 but numerically rank 3, so the rule kept the partition on in exactly the case where it must go; and the idea of a single global spectral threshold, because a healthy target's weak directions and a collapsing target's overlap, pose-dependent, by a factor of 1.65–3.73.

The results

Controller	Retreat 180°	Dropout spike	2 features	Collapsed target	Clean cost
Classic IBVS	✗ 68.19 m, diverges	✗ 28.7×	✓ 0.9×	✓ conv, 8e-11	baseline
Partitioned (2001)	✓ 0.80 m	✓ 1.1×	✗ 288.4×	✗ floors 7.3e-3	+13%
Adaptive-rank truncation	✗ 13.77 m	✓ 0.9×	✓ 0.9×	✓ conv, 2.4e-5	baseline
Both combined (fixed)	✓ 0.80 m	✓ 1.1×	✗ 11.6×	✗ floors 2.0e-3	+13%
Switched, feature count	✓ 0.80 m	✓ 1.1×	✓ 0.5×	✗ floors 2.0e-3	+13%
Switched, σ_6 spectrum	✓ 0.80 m	✓ 1.1×	✓ 0.5×	✓ conv, 2.4e-5	+13%
Switched, area guard	✓ 0.80 m	✓ 0.5×	✓ 0.5×	✓ conv, 2.4e-5	+13%

Read it **down a column, never across one**: retreat and collapse are judged on convergence, the other two on the velocity spike (chapter 9). A single combined score would hide exactly what is being measured.

What the table says:

- Switching **keeps everything** the fixed combination had and closes the two-feature gap from 11.6×
- The only column that separates the switching rules is **collapsed target**, where counting features sees six healthy markers and fails while both geometric signals succeed.
- The cost of switching is zero in the clean case: +13% is the partition's own cost, paid by every row that uses it.

What this killed

The project's founding hypothesis was that this decision needed a learned policy, with a bar set in advance at 2×. A one-line comparison reached 0.5×, so the learning plan was abandoned before any data was collected.

That is worth saying carefully in an interview, because it sounds like an excuse for not doing the hard thing. It is the opposite: the hard thing was building four honest failure cases and six candidate rules, which is what made it possible to *know* that the simple answer was sufficient. Without the comparison you could ship a learned policy and never find out it was unnecessary.

Say it like this

Instead of choosing a control law at design time, choose every step: run the 2001 partition when it's safe and the plain truncated law when it isn't. I implemented six candidate rules for that decision and compared them on all four failures with the measure appropriate to each. Switching closes the worst gap from 11.6× to 0.5×, and the only case that separates the rules is the one where all six markers are visible and the geometry has gone flat.

17. The guard that shipped

In plain words

The 2001 controller leans on one measurement: the size of the patch the markers cover. The guard's job is to notice when that measurement has stopped being trustworthy.

How do you know what trustworthy looks like? Measure it in advance on a part you know is good. Put the camera in a few thousand reasonable positions around it, record the patch size each time, and take a value near the bottom of that range as the alarm line. In operation, if the patch falls below the line, the controller's shortcut is no longer safe.

One number, one comparison, per step.

What it computes

```
sigma = sqrt(polygon area of the visible markers)      # shoelace formula
margin = sigma / threshold_for(number_of_markers_seen)
decision = margin > 1
```

The API returns all of it, not just the boolean, because a decision you cannot inspect is a decision you cannot debug:

```
GuardReading(signal, threshold, margin, n_features, decision)
```

The shoelace formula computes a polygon's area from its corners in order — and "in order" is load-bearing. A self-crossing order gives a small area and fires the guard for a reason that has nothing to do with the part. Chapter 22 is how the ROS node deals with that when a real detector hands over points in arbitrary order.

Calibration, as a procedure

`blindspot/calibrate.py` draws random healthy poses around the goal:

- translation: x, y uniform within a lateral range, z uniform within the working distance range;
- rotation: a random roll about the optical axis, then small random tilts (± 0.3 rad by default);
- any pose putting a marker behind the camera is rejected and redrawn.

It records σ at every pose and takes the **1st percentile**: 4000 poses, fixed seed, reproducible. That percentile is a false-alarm budget — at 1%, the guard gives up the partition on one per cent of perfectly healthy views, which costs almost nothing because the fallback is itself a competent controller.

Three properties of the procedure matter more than the number it produces.

Per target. Healthy 1st percentile: **3.87e-3** for the six-point ring, **1.74e-2** for the four-point square — 4.5× apart, because the statistic carries target scale and working distance. No global constant exists, which is why `FeatureGuard(None)` raises an error

containing the calibrate command rather than shipping a default.

Per visible count. A subset's polygon is always smaller than the full set's, so comparing a 3-marker measurement against a 6-marker threshold is unfair by construction. Calibrated at six and operated at three, the dropout case clears its threshold by only **1.19×**; conditioned on the count, the same instant reads **45.9×**. Below three markers there is no polygon at all, so the threshold at two is exactly 0 and the guard always fires.

On a known-good part, never in situ. This is the deployment trap with no error message. Calibrating on whatever the camera happens to be looking at makes the degeneracy the norm. Measured on the collapsed target: known-good threshold **7.96e-2**, in-situ threshold **1.13e-2**, a factor of seven. With the good threshold the guard fires and the run converges ($2.7e-5$); with the in-situ one the partition stays on for all 600 steps and the run floors at $2.7e-3$. The guard is silently disabled and everything still looks like it is working.

Why this signal rather than the matrix

The design claim: **guard the features your controller is built on, not the matrix it inverts.** The partition's substitute for depth is the area, so the area is what must be healthy for the partition to be safe.

Supporting measurements:

- **Cheap:** a shoelace area over points already in hand, no SVD.
- **Steady:** an area is an aggregate over every visible point, so independent per-point noise largely cancels. 2 px of per-point noise moves it by **0.33%** of its threshold on the square and **0.76%** on the ring, against **1.03%** and **2.35%** for σ_6 — about 3× steadier.

And the honest counterweight, measured later and now in the README: in closed loop σ_6 produced **no** noise-induced switch at all, while the area guard's noise-induced switch on the retreat case is a real dip in area that is not a dip in target health (chapter 25). The steadier statistic did not produce the steadier decision.

Chatter, and why there is no hysteresis

A guard that flickers is worse than no guard. Measured across all four cases at 0, 0.5 and 2 px of feature noise: **no spurious flips at 0 and 0.5 px, and one isolated single-step blip at 2 px**, on the retreat case only.

Hysteresis — requiring the decision to persist for a few steps before acting — was implemented, measured, and **rejected**. It suppresses that blip, but it delays *both* edges, and the edge that matters is dropping the partition the instant the degeneracy appears. On the two-feature case:

hold-off	spike
none	0.5×
2 steps	3.0×
3 steps	3.7×

hold-off	spike
5 steps	5.7×

The cost of a late drop dwarfs the cost of one spurious blip. The parameter still exists in `run_switched` so the result can be re-checked rather than believed.

What the guard does not do

It decides *which law to run*. It does not invent a law for geometry that carries no information.

Three configurations in the study cannot be finished by any controller: permanently two markers, three permanently collinear points, and a goal on the three-point danger cylinder. In all three, classic IBVS, truncation and the guard-switched controller land on the **identical** final pose error to four decimal places (0.0695 m, 0.1130 m). The residual is not something a better controller recovers; it is information the features never carried, and the missing capability is perception or planning.

Saying that is part of the product. A guard that implies it can rescue anything will be trusted where it cannot.

Say it like this

The guard watches one number: the area of the polygon the markers make, which is exactly the quantity the 2001 law uses as a stand-in for distance. The alarm line comes from calibration on a known-good part — thousands of healthy viewpoints, first percentile, per target and per visible count, because a subset's polygon is structurally smaller. Calibrate it in situ and you silently switch it off: I measured the threshold coming out seven times lower, after which it never fires again.

Part 4

Making sure it is true

18. How correctness was established

In plain words

In this kind of work, the code usually runs fine while being completely wrong. A flipped sign doesn't crash anything; it just makes the robot turn the wrong way, and then you spend a day blaming the control theory.

So the rule for the whole project was: **don't reason about whether the maths is right, measure it**. If a formula says "when the camera rotates this way, the dot moves that way", then nudge the camera a tiny bit in simulation, see which way the dot actually moved, and compare. That's a finite-difference check, and it caught two sign bugs that reading the code did not.

Finite differences: `fd_check.py`, 16 checks

The idea: a derivative is a prediction about what happens under a small change. So make the small change and compare.

```
predicted = L @ v          # what the interaction matrix says will happen
actual    = (features after moving by v·h - features before) / h
```

If those disagree beyond numerical noise, something is wrong — and it doesn't matter whether the mistake is in the formula, a sign convention, or the frame you assumed.

The 16 checks cover every derivative and sign the project relies on: each row of the interaction matrix, the derivative of the polygon area feature, the derivative of the line-angle feature (including the exact $d(\alpha)/d(wz) = -1$ that fixes the sign in the partitioned law), `se3_exp` and `se3_log`, and the null spaces claimed for degenerate configurations.

Two sign bugs in `partitioned.py` were found this way, and neither would have been caught by inspection. That sentence is in the README's reproduce section as a standing warning to future-me.

The regression suite: `tests.py`, 177 checks

Every result that took work to establish is locked by a test, so that a later "harmless" change cannot quietly undo it. The suite is organised in numbered sections that mirror the story:

section	what it locks
1	the maths: projection, interaction matrix, SE(3)
2-5	the four failures with their measured numbers, and that truncation is free when nothing is degraded
6	the switched controller and that it doesn't chatter
7-9	σ_6 as a health metric, the fourth case, and that the spectral rule doesn't lose the first three

section	what it locks
10-13	the area guard, where the two signals differ, count-conditioned calibration, the τ artifact
14-15	flips under feature noise; hysteresis tested and rejected
16	the calibration trap
17	forward prediction (dead claim 6)
18	where the controller stops and observability begins
19	the shipped τ 's constructible worse-than-nothing case
20-21	obliquity vs degeneracy, and the retreat row from an imperfect start
22	the folding part, using the arm's own control law

Plus `test_blindspot.py` (34 checks) for the packaged API and the ROS bridge — including tests that the library **refuses** to do things: no default threshold, no guessing units, no accepting a 2-feature polygon as healthy.

A test suite that only checks successes is half a suite. Several of these check that a wrong thing stays wrong: that hysteresis still costs what it cost, that the in-situ calibration still disables the guard, that the τ case still stalls.

The discipline behind the numbers

Fix the seed, report the sampling. Thresholds are the healthy 1st percentile over 4000 random poses with seed 0. Anyone can rerun it and get the same number.

Define the measure before quoting it. "Clean cost +13%" is meaningless until you say "steps to drive the error below 1% of its initial value"; the same comparison gives +2.4% at a 50% threshold.

Report the operating point, not a convenient one. An earlier version of the correlated-noise table quoted the *healthy-phase* margin (1.13×) as though it were the margin during the degradation, which made every row look identical because it was the same quantity measured on the same pre-degradation trajectory. The corrected table reports the margin at the count actually seen that step (1.62×). That correction is written into the README rather than silently fixed.

When a result surprises you, suspect the instrument first. Three of the five rendered-camera bugs in chapter 20 were found because a number was *too good*, not too bad.

Say it like this

I verified derivatives by finite difference instead of reasoning about sign conventions, because two sign bugs in the partitioned law had already hidden from inspection. Everything that took effort to establish is locked by a regression test — 177 of them, plus 34 on the packaged API — including tests that assert the things that didn't work

still don't, so a future change can't quietly revive a claim I killed.

19. Dead claims

In plain words

This is the chapter to read before an interview. It is a list of things I believed, wrote down, and then had to take back because a measurement said otherwise. Each one has the number that killed it.

Keeping this list is not modesty. In work like this, the claims that survived are only trustworthy if you can see how many didn't.

1. "Closing the 11.6× gap needs a learned policy"

The project's founding hypothesis.

Killed by a one-line hand-written rule reaching **0.5×**, against a bar of 2× that was set before running it. No learned policy is needed for this decision.

The reason this is a *useful* negative result: training a policy would have meant collecting data, defining a reward, and shipping a model that nobody can inspect — to replace a comparison between two numbers.

2. "A rank-margin rule on the spectrum beats counting features"

Killed by finite difference. At two features, L_{xy} is 4×4 but **numerically rank 3**, so the condition $\text{rank}(L_{xy}) < \text{rank}(L)$ held and the partition stayed ON in exactly the case where it must be dropped.

The bug was comparing against the wrong matrix. The fixed rule compares the numerical rank of the *full* L against the number of unknowns in the reduced solve.

3. "A single global τ separates healthy-but-weak from degenerate"

Killed by the measured band. A healthy ring's weak directions sit at $\sigma_5/\sigma_1 = \mathbf{1.4e-3} \dots \mathbf{3.1e-3}$ depending on pose. A *collapsing* target's σ_5 is already at **8.2e-4** — below healthy. The separating band is a factor of **1.65-3.73**, and it moves with pose.

So no global threshold is defensible. A rank test has to use an existence threshold instead, after which it reduces to counting features, which is already a rule we have.

4. "The switching decision needs the interaction-matrix spectrum"

Killed, then partly revived. This is the most instructive entry in the project, because it moved twice.

Killed: on all four benchmark cases the area guard matched σ_6 everywhere and beat it on dropout (0.5× vs 1.1×), with no spectrum at all.

Revived: that conclusion was a property of the **test suite**, not of the signals. None of the four cases separates them. The Gazebo arm work found one that does — a healthy planar target viewed obliquely:

	area margin	σ_6 margin
healthy, face-on	1.69×	1.88×
healthy, 70°	0.99× fires	27.01×
healthy, 75°	0.86× fires	28.72×
collapsed in 3D, $c = 0.5$	1.19×	0.74× fires
collapsed in 3D, $c = 0.05$	0.38×	0.07×

The area signal cannot tell a degenerate part from a healthy one seen at a steep angle; both shrink the projected polygon. σ_6 can, and in the direction physics predicts: a face-on planar target is the *worst*-conditioned IBVS case because every point sits at one depth, so tilting it **improves** conditioning (σ_6 rises about 15×) while the projected area shrinks.

And then the cost was measured, which is chapter 25: on a tilting target the false positive is free, but on the retreat benchmark from a start just 2 cm or 3° off the exact symmetric one, it costs a 2.0–2.9 m retreat in 100 runs out of 100 — while σ_6 , by staying silent, rides the partition into losing the target entirely in 3 runs of 100.

Final ledger: neither dominates. σ_6 is the faithful signal of whether the *matrix* is degenerate; the area is the faithful signal of whether the *partition's own feature* is healthy, and the folding part (chapter 26) is the case where that difference saves the run and σ_6 is blind at 1.29×.

5. "The collapsed-target velocity spike is evidence for switching"

Killed by τ arithmetic. At $\tau=1e-3$ the truncation threshold sits at **98.78%** of the smallest singular value, so the spike is a 1.2%-margin artifact: across the τ sweep, combined's peak falls **5.96** → **0.326** against truncation-only's **0.322**. The gap essentially vanishes.

Only the *convergence* half of that row is real evidence, and it is τ -independent from $1e-4$ to $1e-1$. The results table therefore reports that case on convergence and says in the caption why.

6. "Predicting the guard signal forward buys useful warning"

This one died on structure, not on quality. Rolling the features forward with $s_{next} = s + L \cdot v \cdot dt$ predicts the degeneracy the **camera** causes, never the one the **world** causes:

transition	example	warning
camera-driven	classic IBVS retreating	231 ms
exogenous	an occluder arrives	0 ms
already degenerate at $t=0$	collapsed target	0 ms

The predictor itself is good — one step is sub-pixel (0.157 px median), and 20 steps (660 ms) stays trustworthy even at high speed — and it costs 1.394 ms per cycle, so the gate passed. It dies because with the partition on, the camera never retreats, so the area never falls, so **the one case with warning is the case that never needed it**. And with 231 ms in hand, ramping the gain down buys 4% on peak velocity and nothing on convergence.

7. "The guard's switch caused the spike in the arm demo"

Mine, written into a resume note before it was measured, and retracted the next day. The tilting-panel arm run showed the guarded cell spiking to 2.561 while the unguarded one stayed at 0.526, and I attributed it to the switch being "a pure discontinuity between two control laws".

Reproduced in numpy across 16 tilt onset/speed combinations: the guard fires every time, and **peak commanded velocity was identical under all three rules in 15 of 16** (the 16th differed by 5%). Switching is nearly free there. The single arm spike never reproduced and is more likely perception noise at a grazing view.

The correction is written into the repo where the wrong claim was, not silently deleted.

Two further corrections

- **The health metric $S[-1]$ was wrong.** It compares matrices of different shapes and cannot see a null space: on a two-feature L it reads a healthy **9.97e-2** where the padded σ_6 is **exactly 0**.
- **The collapsed-target mechanism is not $\sigma = \sqrt{\text{area}} \rightarrow 0$.** σ and σ^* shrink together, the ratio holds at **1.4980**, and v_z is unchanged at every collapse level. The failure is in the reduced inverse.

Say it like this

Seven claims of mine died in this project, including the founding hypothesis, and each one is in the README with the measurement that killed it. The most useful one moved twice: I killed the idea that the decision needs the matrix spectrum, then the arm demo found a case my four-case test suite didn't contain, which partly revived it. That taught me the failure wasn't in the signal, it was in my test set.

20. A real camera and a real detector

In plain words

Up to here, the "camera" was maths: I knew exactly where every marker was, because I computed it. Real cameras don't work like that. They render pixels, and a detector has to find the markers in those pixels, with blur, lighting and rounding errors.

So the next step was to replace the perfect camera with a rendered one (MuJoCo, a physics and graphics engine) showing real ArUco markers — the black and white squares you've seen on robots — and let OpenCV's real detector find them. Then rerun the conclusions and see which survive.

Three of the four verdicts reproduced. The fourth turned out to be **structurally untestable** with markers, for a reason worth understanding.

The gate

Before trusting any conclusion from rendered images, one number had to be established: does the rendered-and-detected feature position agree with the exact projection?

0.095 px mean. That is `mj_validate.py`, and it is a gate, not a result: if this had been 2 px, nothing downstream would have meant anything.

Five bugs the rendered port surfaced

Every one was found by measurement, not by reading code. Two of them produced confident wrong conclusions that were written down before being caught.

1. The half-pixel principal point, and the refinement that hid it

Detected positions carried a systematic bias of exactly **(−0.4904, −0.5010) px**. Not noise — a bias, the same in both axes, suspiciously close to half a pixel.

It is half a pixel. OpenGL maps its normalised coordinates onto a **continuous** pixel range, so the optical axis lands at $W/2$. OpenCV indexes pixel **centres**, which sit half a pixel lower, so the principal point is $(W-1)/2 = 639.5$, not 640. Fixing it dropped the bias to $(+0.0096, -0.0010)$ px.

The part worth remembering: before the fix, three corner-refinement methods were compared, and **APRILTAG looked by far the best** (0.127 px against 0.72). It was not better. Its corner convention is offset by about half a pixel in the *opposite* direction, so it was silently cancelling the bug:

refinement	before fix	after fix
NONE	0.7208 px	0.1990 px
SUBPIX	0.7145 px	0.0836 px
APRILTAG	0.1266 px	0.5972 px

Two errors of opposite sign, each hiding the other. Picking the method by measured accuracy alone would have locked in **both**. The lesson generalises: a systematic offset is evidence about a *convention*; a mean error is not.

2. Mirrored markers

The camera views the target plane from its back side, and a marker seen from behind is mirrored. ArUco dictionaries are not mirror-invariant: **0 of 6 detected as rendered, 6 of 6 after mirroring the texture**.

Fixed by mirroring the texture, not by flipping the captured image — flipping the frame would have hidden a genuine geometry error instead of fixing one.

3. MuJoCo's auto-computed `extent` clipped the scene

Near and far clipping planes are derived from a model "extent" that MuJoCo computes from the geometry. Shrinking the markers therefore moved the near plane and clipped everything: **image mean fell from 254 to 13 with no error raised**. Rendering must not depend on the geometry under test, so `extent` is now pinned.

4. Live renderers degrade silently, and faked a physical limit

Open renderer instances accumulate GPU resources. Past a handful, frames come back **visually plausible but undetectable**. A sweep that built ~30 renderers in one process reported 0 of 6 detections for every scene after the first.

That looked exactly like a physical limit of the geometry, and **was reported as one**: "collapse is unrenderable". It was not. Closing each renderer showed collapse renders cleanly down to $c = 0.06$. The real limit had to be established a second time, by arithmetic on both constraints rather than by a sweep.

5. Ghosting in the motion-blur model

Accumulation blur with a fixed, small number of sub-frames places samples further apart than a marker, producing a row of ghost copies instead of a smear. The detector locked onto a ghost and reported a feature **156 px** from the truth — and detection *improved* with more blur, which is the signature of an artifact, since blur cannot help. The sub-frame count now adapts so successive samples land under a pixel apart.

Correlated degradation: the honest hard case

Independent random noise is the easy case, and the guard shrugs it off ($d' = 454$ at 0.1–0.5 px; still 7.1 at a catastrophic 8 px; 15 with 20% gross outliers). The hard case is noise that **co-varies with the failure**: a control lurch causes motion blur on the same step, which corrupts the very measurement that would have detected the lurch.

That needs a rendered camera to study at all. `mj_degrade.py` drives motion blur from the camera's own measured inter-frame motion and adds a lagged auto-exposure chasing image brightness. Measured at 10 fps with $\lambda = 1.5$:

blur across exposure	N visible	operating margin	detector failure	run
0 px	3	1.62×	0.00%	converges
5.1 px	3	1.62×	0.00%	converges
7.6 px	3	1.63×	0.42%	converges
12.7 px	0	—	50%	target lost

Markers are ~ 41 px, so the cliff sits near a quarter of a marker width. The finding: the guard's margin is **flat right up to the cliff and then its input vanishes**. Correlated degradation does not *bias* the guard, it *removes* its input — so a guard failure and a detection failure stay cleanly distinguishable, which is the property you want when something goes wrong in a real cell.

The case markers cannot test

The collapsed target is a degeneracy of *2D extent*, and an area-based marker needs 2D extent to decode. Reproducing the numpy verdict needs collapse $c \leq 0.05$; the smallest collapse leaving six decodable markers at this working distance is $c = 0.35$, even at 2560 px wide. A factor of seven, and **not a resolution limit** — the guard's trigger condition and the detector's failure coincide exactly in that regime.

That is why the Gazebo work later used plain dark dots rather than ArUco markers, and why the README says real deployments need corner or edge features there.

Say it like this

I replaced the perfect camera with a rendered one and OpenCV's real ArUco detector, gated on agreeing with exact projection to 0.095 pixels. That port surfaced five bugs, and two of them had already produced confident wrong conclusions — including one where a corner-refinement method looked best only because its convention was cancelling a half-pixel error of mine. Three of the four verdicts reproduced; the fourth is untestable with markers for a structural reason, which is itself a result.

Part 5

On a robot

21. ROS 2, from zero

Everything from here on runs on ROS 2, so this chapter explains it from nothing. If you already write ROS nodes, skip to the last section, which is what this project specifically uses.

The problem ROS solves

A robot is many programs at once: one reads the camera, one finds markers, one decides what to do, one talks to the motors, one logs everything. They start and stop at different times, some crash, some run on a different machine.

Wiring them together by hand — sockets, file formats, start-up order — is a job nobody wants to do twice. **ROS 2** (Robot Operating System 2) is the standard plumbing that does it for you. It is not an operating system; it is a set of libraries and conventions.

The five ideas

Node. One program that does one job. `guard_node` is a node.

Topic. A named channel. Nodes **publish** to it, other nodes **subscribe**. Nobody needs to know who else is connected, so you can add a logger or a visualiser without touching anything.

Message. The typed shape of what travels on a topic. `sensor_msgs/Image` is a picture, `std_msgs/Bool` is a true/false, `vision_msgs/Detection2DArray` is a list of detections. Types are shared across the whole ecosystem, which is what makes independently written nodes interoperate.

Parameter. A named setting a node reads at start-up — a file path, a threshold, a rate.

Launch file. A Python script describing which nodes to start, with which parameters and remappings, and in what order. One command brings up the whole system.

Two more that this project needs:

TF. The transform tree: a live record of where every frame is relative to every other (chapter 2's poses, published continuously). Ask "where is the camera in the robot's base frame right now?" and TF answers.

Simulation time. When a simulator is running, it publishes a clock on `/clock`, and nodes started with `use_sim_time` follow *that* clock instead of the wall clock. It matters more than it sounds: if the simulator runs at a quarter of real speed because it is rendering 1080p video, a node on wall-clock time silently runs its control loop four times too often per simulated second. That was a real bug in this project (chapter 23).

Names, namespaces and remapping

Topics are strings, so two copies of the same node would collide. **Namespaces** prefix them: the same code launched under `/left` and `/right` publishes `/left/joint_states` and `/right/joint_states`. **Remapping** rewires a topic name at launch time without editing code, which is how the guard node listens to your detector's topic rather than a hard-coded one:

```
ros2 run blindspot_ros guard_node --ros-args \
  -p calibration:=/abs/path/my_target.json \
  -r detections:=/aruco/detections
```

ros2_control, in one page

Motors need commands at a steady, high rate — hundreds of hertz — with a consistent interface across different hardware. `ros2_control` is the framework for that.

- A **hardware interface** exposes a robot's joints as **command interfaces** (what you can set: position, velocity, effort) and **state interfaces** (what you can read).
- A **controller** turns a higher-level goal into those commands. This project uses `JointTrajectoryController`, which accepts a list of joint positions with times and interpolates between them, and `JointStateBroadcaster`, which publishes the measured joint positions.
- The **controller manager** loads, starts and stops controllers, and runs them in a fixed-rate loop (200 Hz here).

In simulation, the hardware interface is provided by `gz_ros2_control`, which plugs the same machinery into Gazebo. That is the property worth having: the control side of the stack does not know it is in a simulator.

Gazebo and the bridge

Gazebo simulates physics and renders sensors. It has its own message system, so `ros_gz_bridge` translates between the two worlds in whichever direction you specify:

```
/clock@rosgraph_msgs/msg/Clock[gz_msgs.Clock      # gz -> ROS
/fold_left@std_msgs/msg/Float64[gz_msgs.Double    # ROS -> gz
```

The bracket points the way the data flows. Anything not bridged simply does not exist on the ROS side, which is one of the ways an afternoon disappears.

What this project uses

piece	where
nodes, topics, parameters, namespaces, remapping	everywhere
vision_msgs, sensor_msgs, std_msgs, diagnostic_msgs	the guard node (chapter 22)
trajectory_msgs, visualization_msgs	the arm and the RViz simulation

piece	where
launch files with staged actions	both demos
TF	verifying hand-written kinematics (chapter 24)
RViz	the live guard simulation
<code>ros2_control</code> + <code>gz_ros2_control</code> + <code>JointTrajectoryController</code>	the arm cells (chapter 23)
<code>ros_gz_bridge</code> , <code>ros_gz_sim</code>	clock, fold commands, camera images, spawning
simulation time	every node in the arm demo

Two packages hold it: `blindspot_ros` (the guard node, a fake-detection publisher, and an RViz simulation) and `blindspot_arm` (the Gazebo cells, the control law, the recorder).

Say it like this

ROS 2 is the plumbing: independent programs called nodes exchange typed messages over named topics, started together by a launch file, with parameters and remapping so the same code runs in two namespaces. On top of that, `ros2_control` owns the joint-level loop and `gz_ros2_control` gives the same interface in Gazebo, so the control side never knows it's in a simulator. And everything runs on the simulator's clock, not the wall clock.

22. Packaging it: the library and the ROS 2 node

In plain words

A result in a script is not something anyone can use. Two things had to exist before this could be called finished: a small library someone can import into their own control loop, and a **ROS 2 node** that publishes the decision so a real robot stack can consume it.

ROS 2 is the standard plumbing for robots: independent programs called **nodes** talk to each other by publishing and subscribing to named **topics**, with typed **messages**. The guard is a node. It reads detections and camera calibration, and it publishes one boolean.

The library contract

```
from blindspot import FeatureGuard
guard = FeatureGuard.load("my_target.json")

if guard.partition_ok(s_visible):          # normalised image coordinates
    v = my_partitioned_control(...)
else:
    v = my_plain_control(...)

h = guard.evaluate(s_visible) # signal, threshold, margin, n_features, decision
```

Four design rules, all of them refusals:

1. **No default threshold.** `FeatureGuard(None)` raises, with the `calibrate` command in the error text and the reason: the threshold carries target scale and working distance (measured $3.9\text{e-}3$ for one target, $1.7\text{e-}2$ for another).
2. **No unit guessing.** The guard works in normalised image coordinates. `blindspot/units.py` converts from pixels and *requires* the principal point; it will not assume $w/2$ (chapter 20 explains why that half pixel matters).
3. **No controllers in the product.** The verified control laws live in `blindspot/reference/`, whose docstring says they are "EXAMPLES, not the product". The shipped thing decides; it does not drive.
4. **No hidden state.** `evaluate()` returns the signal, the threshold, the margin, the feature count and the decision, so a log line explains itself.

`test_blindspot.py` has 34 checks, including checks that these refusals still refuse.

The ROS 2 node

Package `blindspot_ros`, node `guard_node`:

direction	topic	type
in	detections	vision_msgs/Detection2DArray

direction	topic	type
in	camera_info	sensor_msgs/CameraInfo
out	~/partition_ok	std_msgs/Bool
out	/diagnostics	diagnostic_msgs/DiagnosticArray

```
ros2 run blindspot_ros guard_node --ros-args \
  -p calibration:=/abs/path/my_target.json \
  -r detections:=/aruco/detections \
  -r camera_info:=/camera/camera_info
```

It does **not** control anything. Keep your servo loop, gate it on the Bool. That is a deliberate boundary: a node that both decides and drives is a node nobody can adopt incrementally.

Three things the node handles that are easy to get wrong in a real stack:

Ordering. The guard signal is a shoelace polygon area, so it depends on the order of the points — and a detector reports whatever order it happens to find. An order that crosses itself gives a small area and fires the guard for a reason that has nothing to do with geometry. So detections are ordered **by marker id**, which reproduces the order calibration used; without ids there is an angular fallback; and the ordering actually used is published in the diagnostic. The node also cross-checks the shoelace area against the convex hull area and raises the diagnostic to **ERROR** when they disagree, because at that point the decision is not measuring what it claims to.

Intrinsics. From `camera_info` when the camera is calibrated. If it publishes zeros, set `fovy_deg` and the node falls back to a principal point of $(W-1)/2$ — and says in the diagnostic which path it used.

Refusing to start. `calibration` is a required parameter. Without it the node exits immediately and prints the calibrate command. Same reason as the library: a default threshold would be a silent wrong answer.

All the ordering, id and unit logic lives in `blindspot/ros_bridge.py`, which imports **no ROS at all**, so it is unit-tested on a machine with no ROS installed. The `rcldpy` node is a thin wrapper over it.

Verified running

`demo.launch.py` starts the guard plus a `fake_detections` node that needs no camera and deliberately **shuffles** the detection order every frame, so the ordering path is exercised rather than assumed:

phase	n	margin	decision	diagnostic	ordering
healthy, 6 markers	6	1.564	partition_ok	OK	id
occluded, 2 markers	2	0.000	drop_partition	WARN	id

phase	n	margin	decision	diagnostic	ordering
collapsed target, 6 markers	6	0.221	drop_partition	WARN	id

Those margins reproduce the library's own numbers (1.55 and 0.22 in `examples/quickstart.py`), so the ROS path is not computing something different. The third row is the case feature counting cannot see: six markers present, and the guard correctly drops the partition anyway.

The live simulation

`sim.launch.py` opens RViz and runs a servo loop **closed through the guard node**: the control law is chosen every step by the Bool on `/blindspot_guard/partition_ok`.

```
partition_ok == True    -> the 2001 partitioned law
partition_ok == False  -> plain truncated law on the full interaction matrix
```

`sim_node` publishes `camera/image`, `markers` (a `MarkerArray` showing the target and the camera frustum in 3D), `tf`, `detections` and `camera_info`, and cycles healthy → occluded to two markers → collapsed target, re-posing the camera at each change so every regime shows a real approach rather than a converged still. The images are drawn with numpy, so there is no OpenCV or `cv_bridge` dependency. `rviz:=false` runs it headless.

A bug that only the real distro found

Verified on **ROS 2 Lyrical** (Ubuntu 26.04, system Python 3.14, numpy 2.3.5) and also on Jazzy via RoboStack. Running it on the real distro immediately found a bug Jazzy did not: `rclpy` on Lyrical shuts the context down inside its own SIGINT handler, so `spin()` raises `ExternalShutdownException` rather than `KeyboardInterrupt`. Catching only the latter exits with status 1 on a clean Ctrl-C. All three nodes now catch both.

Being distro-agnostic in principle was an argument. The argument was wrong, and one apt install settled it.

Say it like this

The deliverable is a small library plus a ROS 2 node that publishes one boolean on a topic, so it drops into someone else's servo loop without taking it over. The node handles the things that actually bite in a real stack: detection ordering, because the signal is a polygon area and a detector returns points in any order; intrinsics from camera_info with a stated fallback; and refusing to start without a calibration file, because a default threshold would be a silent wrong answer.

23. Building the Gazebo cell

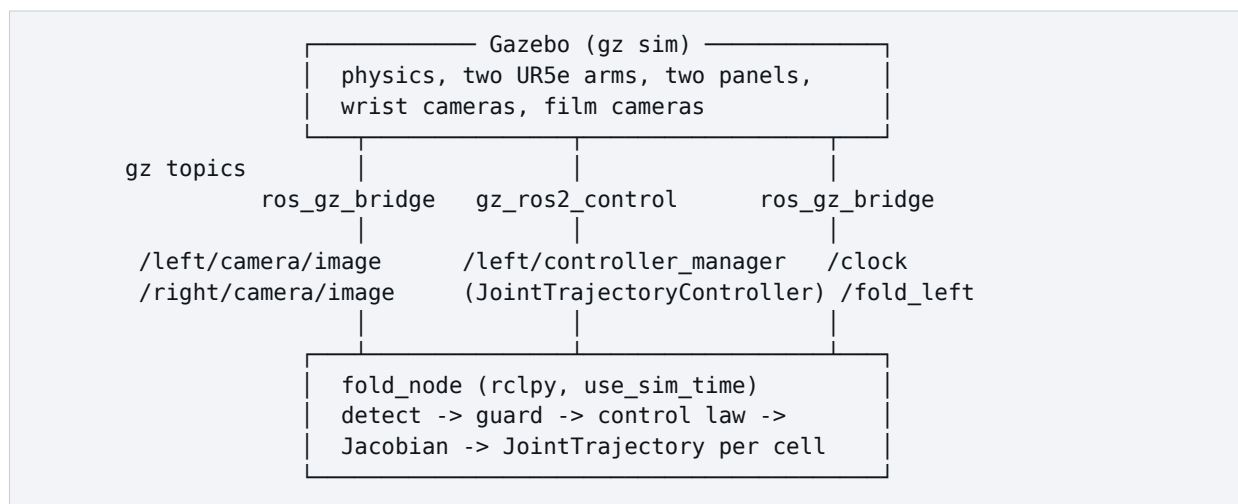
In plain words

Everything so far assumed a camera that could fly anywhere. A real robot cannot: it has joints with limits, a controller that takes position commands, and a camera bolted to its wrist.

So the last phase put the whole thing on a simulated **UR5e** industrial arm in Gazebo, driven through ROS 2, with the guard choosing the control law every step. And to make the comparison airtight, **two** arms in **one** world: the left one always runs the 2001 partition, the right one obeys the guard.

One world means one physics clock, so the two cells are frame-locked by construction. Two separate simulators would drift apart and the comparison would be worthless.

What is running



The robot description

`ur5e_camera.urdf.xacro` wraps the standard `ur_description` macro and adds:

- a **camera link** on `tool0` carrying a Gazebo camera sensor: 640×480, 60° horizontal field of view, 20 Hz;
- an **optical frame** child link in the computer-vision convention (+z forward, +y down) — the frame the guard's normalised coordinates live in;
- a `ros2_control` block whose hardware plugin is `gz_ros2_control/GazeboSimSystem`, declaring a **position** command interface and **position, velocity, effort** state interfaces per joint;
- the `gz_ros2_control` system plugin with `<ros><namespace>`, so the two cells get separate controller managers.

`xacro` is a macro language for URDF: the file takes `prefix` and `ns` arguments, so the same description spawns `left_shoulder_pan_joint` under `/left` and `right_...` under `/right`. One file, two cells, no copy-paste and no chance of them differing.

The controllers

```
/left/controller_manager:
  ros__parameters:
    update_rate: 200
    joint_state_broadcaster: {type: joint_state_broadcaster/JointStateBroadcaster}
    arm_controller:          {type: joint_trajectory_controller/JointTrajectoryController}

/left/arm_controller:
  ros__parameters:
    joints: [left_shoulder_pan_joint, ...]
    command_interfaces: [position]
    state_interfaces: [position, velocity]
    allow_nonzero_velocity_at_trajectory_end: true
```

The servo loop sends a **single trajectory point** each cycle: the next joint configuration, to be reached in 0.16 s. That is a simple, honest way to drive a position-controlled industrial arm from a 10 Hz vision loop, and it is close to what a real cell does.

The world

`panel_fold.sdf` holds, per cell: a post anchored to the world, a light backdrop, and two flaps on hinges carrying the markers, driven by joint position controllers on a ROS topic. Plus two **film cameras** — static camera sensors used only for recording the clip, never by the controller.

Design choices that each fixed a specific problem are in chapter 26, because they belong to the folding scenario.

The launch file

`fold.launch.py` starts, in order:

1. `gz sim` with the world, and `-z <rate>` to set physics steps per wall second (this is how recording runs are slowed down);
2. a `ros_gz_bridge` for `/clock`, for the fold commands (`std_msgs/Float64` → `gz.msgs.Double`), and for the film cameras;
3. per cell: `robot_state_publisher` in its namespace with `use_sim_time`, a `ros_gz_sim create` that spawns the robot from its `/robot_description` topic, a camera bridge with remappings to `/left/camera/image`, and two `controller_manager` spawners.

The spawners are staged with `TimerAction` at 6 s, 10 s and 13 s, with `--controller-manager-timeout 120`, because the controller manager does not exist until the model is spawned and the plugin has loaded.

Simulation time, and the bug that proved it matters

Every node runs with `use_sim_time`. The film cameras render 1080p at 60 Hz, which drops the simulation to roughly a quarter of real time — and that must not change the control loop. On sim time it does not: only wall-clock duration changes.

The first version used a wall-clock timer, so turning recording on silently changed the controller's effective rate. Nothing crashed. The run just meant something different from the run before it, which is the worst kind of bug in a measurement project.

The problems, in the order they cost time

Environment

- A shell that has sourced a conda ROS environment feeds conda libraries to system binaries and breaks them confusingly. Both run scripts start by clearing `LD_LIBRARY_PATH`, `PYTHONPATH`, `CMAKE_PREFIX_PATH` and `AMENT_PREFIX_PATH`.
- Gazebo could not find the world's models or the `gz_ros2_control` plugin until `GZ_SIM_RESOURCE_PATH` and `GZ_SIM_SYSTEM_PLUGIN_PATH` were set in the launch environment.

Launch and controllers

- `robot_description` must be passed as `ParameterValue(..., value_type=str)`, or the launch system infers the wrong type from the XML.
- Controller spawners raced the controller manager. Fixed with staged `TimerActions` and a long timeout rather than a sleep and hope.

The world

- The panel model cannot be `<static>` because its joints must move, so gravity dropped it from $z = 0.359$ m to 0.085 m, out of the camera's view entirely. Fixed with an explicit fixed joint anchoring the post to world.
- Prefixing the robot's `world` link broke that anchoring, because the joint then referred to a link that no longer existed under that name.

Process hygiene (the ones that waste whole evenings)

- `kill -f <pattern>` matches the running shell's **own** command line, so a cleanup line kept killing the session that issued it. Bracketed patterns (`[d]uel_node`) fix it.
- Gazebo's process on this version is `gz-sim-main`, not `gz sim`. Stale servers from earlier runs stacked up and silently corrupted measurements until every run was isolated on its own `ROS_DOMAIN_ID` and the cleanup was made reliable.
- A background job in a non-interactive script **ignores SIGINT**, so the first cleanup handler did nothing at all. And bash defers a trap until the *foreground* command returns, so a node that never exits made `SIGTERM` a no-op too. The run script now starts each child with `setsid` and kills process groups.

None of this is control theory. All of it is what "I ran it on a robot" actually costs, and being able to tell that story is half of what a robotics interview is asking about.

Say it like this

Two UR5e cells in one Gazebo world so they share a physics clock, described by one xacro file instantiated twice under different namespaces, with `gz_ros2_control` giving each a controller manager and a `JointTrajectoryController`. The launch file stages the spawners because the controller manager doesn't exist until the model is up,

everything runs on simulation time so recording can slow the simulator without changing the control rate, and the panel needed an explicit anchor joint because a model with moving joints can't be static and gravity took it out of frame.

24. Kinematics and perception on the arm

Two things stand between the control law and the robot: turning a camera twist into joint motion, and turning an image into marker positions. This chapter is both, and both were written by hand so that they could be checked.

From camera twist to joint commands

The control law outputs a twist for the **camera**. The robot accepts joint positions. Three steps bridge them.

1. Forward kinematics. Where is the camera, given the joint angles? Walk the chain of links from the base to the camera frame, multiplying each joint's 4×4 transform (chapter 2). That is `kin.py`: it parses the URDF with `urdf_parser_py` and composes the transforms in `numpy`.

2. The geometric Jacobian. How does the camera move when each joint moves? For a revolute joint with axis `z_i` (in base coordinates) whose origin is at `p_i`, and with the camera at `p_e`:

```
linear part of column i = z_i × (p_e - p_i)
angular part of column i = z_i
```

Stack the six columns and you have `J`, a 6×6 matrix with `twist = J · q̇`. The intuition for the cross product: rotating about an axis moves a point at distance `r` from it with speed `ω × r`.

3. Invert it. `q̇ = J+ · twist`, then clip each joint velocity to ± 1.5 rad/s, integrate one step (`q_next = q + q̇ · dt`), and publish `q_next` as a single-point trajectory.

Note that this is the *same* mathematics as the visual servo itself — a matrix mapping motion to observation, inverted to get the command — one level down. And it has the same failure mode: near a kinematic singularity `J` becomes ill-conditioned and the joint speeds explode. The velocity clip is the crude protection; a robot cell would use a damped least-squares inverse.

Why hand-written kinematics is defensible here

No KDL, no MoveIt, about 90 lines of `numpy`. The reason it is trustworthy is not that it is short — it is that it is **checked against something independent**: `verify_against_tf()` compares this forward kinematics to what `robot_state_publisher` reports over TF, which is computed by different code from the same URDF.

Agreement: 1e-6 m. A wrong joint axis, a missed fixed joint or a transposed rotation shows up immediately at that tolerance. This is chapter 18's principle applied to robot geometry: do not reason about whether your chain is right, measure it against an independent implementation.

Perception: finding six dots

The markers are plain dark circles on a light panel, and the detector is deliberately simple (`detect.py`):

1. convert to grayscale;
2. threshold: anything darker than 90 is marker-ish;
3. connected components — group touching dark pixels into blobs;
4. keep blobs whose area is between 25 and 4000 px, which rejects specks and large dark regions;
5. from those, pick **six** and return their centroids.

Step 5 is where the interesting bug lived. The obvious rule is "take the six largest", and it fails: at close standoff the robot's own dark wrist enters the frame and is *bigger* than any marker, so it displaces a real one and the controller servos on a lie.

The fix is to pick the six blobs most **similar in area** to each other, because the six real markers are identical circles at similar depths. It is a one-line change and it is the difference between a demo that works and one that fails only at close range, which is the worst time to fail.

Why plain dots and not ArUco

ArUco markers carry ids, which solves correspondence for free. They also need 2D extent to decode — and the whole point of the folding scenario is that the 2D extent collapses (chapter 13's structural limit). At the fold angles that matter, ArUco stops decoding before the geometry is interesting.

Dark dots stay detectable all the way through the collapse. And in the folding world they are **spheres**, not discs, so that a marker on a flap rotated 85° still projects as a circle instead of vanishing edge-on.

The price is correspondence, which has to be solved explicitly.

Correspondence, explicitly

The detector returns six centroids in arbitrary order. To subtract `s*` from `s` you need to know which is which.

Step 1: order by angle. Compute the centroid of the six points and sort them by the angle around it. For points on a ring this fixes the cyclic order regardless of how the detector found them.

Step 2: fix where the order starts. Angular ordering does not say which dot is "first", so a rotated target can pair every dot with the wrong partner. Measured consequence: one cell converged to an error of 0.090 while its identical twin diverged to 0.530, running the same code.

The fix is to try all six cyclic shifts of the desired features and keep the one with the smallest residual:

```
best = min((np.linalg.norm(s - np.roll(s_star, k, axis=0)), k) for k in range(6))
```

Six comparisons per step, and the ambiguity is gone.

Intrinsics, and a stall that looked like a control bug

The node needs focal length and principal point to convert pixels to normalised coordinates. The natural source is `camera_info`, and on this setup it never arrived on the ROS side. Worse, waiting for it stalled the servo loop silently: the HUD read 0/6 features and neither arm moved, which looks exactly like a controller problem.

The fix is to take the intrinsics from the sensor definition in the URDF, where they are already declared, with the $(W-1)/2$ principal point convention from chapter 20 — and to write a comment saying why, so that the next person does not "fix" it back.

The control node, end to end

Per cell, ten times a second:

1. latest wrist camera frame;
2. detect the six dots;
3. order by angle, convert to normalised coordinates;
4. ask the guard for a reading (calibrated at start-up on the known-good hexagon);
5. pick the control law — left cell always partition, right cell obeys the guard;
6. compute the twist with the **same law the numpy study uses**, imported from `law.py`, which `tests.py` also imports, so the tests check the law the arm actually runs;
7. Jacobian, clip, integrate, publish;
8. apply the shared **protective stop** if the next pose would come within 12 cm of the part;
9. log everything to CSV at 10 Hz.

Step 6 is the part worth pointing at in an interview. There is exactly one implementation of the control law in the repository, and both the regression tests and the robot import it. A study whose conclusions are checked against a *copy* of the deployed code is checking the wrong thing.

Say it like this

The control law produces a camera twist, so I map it through the arm's geometric Jacobian — cross product of joint axis with the lever arm, written by hand in numpy from the URDF and verified against TF to a micrometre — clip joint speeds, integrate a step and publish a trajectory point. Perception is deliberately simple: threshold, connected components, and pick the six blobs most similar in area, because picking the six largest lets the robot's own wrist masquerade as a marker at close range. Correspondence is six cyclic shifts, chosen by residual.

25. What the arm found

In plain words

The point of putting it on a robot was to see whether anything broke that a free-flying camera could not have shown. Two things did, and both changed what the README claims.

The first: the guard **cried wolf**. A perfectly healthy panel, tilted so the camera sees it at a steep angle, looks exactly like a collapsing one — because both make the marker patch smaller. The second, found while checking a sentence I had written about noise: the benchmark that made the guard look best is set up in a way that hides a real cost.

Finding 1: the false positive

The first Gazebo demo tilted the panel toward edge-on mid-approach. The guarded cell dropped the partition at margin $0.55\times$ and held its standoff; the unguarded one kept the partition and drove in close.

That looks like a win, and it is not. The tilted panel is **healthy**. Nothing about its geometry is degenerate; it is just being viewed obliquely. The guard fired on a target that was fine.

Reproduced in numpy, on the ring target:

	area margin	σ_6 margin
healthy, face-on	$1.69\times$	$1.88\times$
healthy, 70°	$0.99\times$ — fires	$27.01\times$ — silent
healthy, 75°	$0.86\times$ — fires	$28.72\times$ — silent
collapsed in 3D, $c = 0.5$	$1.19\times$ — misses	$0.74\times$ — fires

And the physics runs the way chapter 7 predicts: a face-on planar target is the *worst*-conditioned case, so tilting it **improves** the matrix — σ_6 rises about $15\times$ — while the projected area shrinks. The two signals move in opposite directions on the same event.

This is why dead claim 4 was revived. My four-case test suite contained no case that separates the two signals, so "they matched everywhere" was a fact about my test set, not about the signals.

The claim I got wrong, and retracted

In the same run, the guarded cell showed a velocity spike of 2.561 while the unguarded one stayed at 0.526. I wrote down that the switch itself caused it: "a pure discontinuity between two control laws".

Then I measured it. Sixteen tilt onset/speed combinations in numpy: the guard fires every time, and **peak commanded velocity is identical under partition-always, area guard and σ_6 in 15 of the 16** (the 16th differs by 5%). With little error left to correct, the two laws compute nearly the same twist, so switching between them costs almost

nothing.

The arm spike never reproduced. The most likely cause is perception noise at a grazing view, where the dots are thin ellipses. The retraction is written into the file where the wrong claim was.

Worth saying plainly in an interview: **the attribution was too quick, and the measurement was what caught it.** That is the same discipline as chapter 18, applied to my own conclusion.

Finding 2: the benchmark's symmetric start

This one came from checking a sentence, not from looking for a bug. The README said the area guard is "more noise-robust" than σ_6 . The statistic part was measured; the comparison to σ_6 never had been. Measuring it turned up something bigger.

The retreat benchmark starts from an **exact** 180° rotation about the optical axis. In that exact configuration, the partition's sideways command is zero by symmetry — measured 2.1e-15, i.e. numerically zero.

Start it 2 cm off, or 3° off, and the symmetry breaks. All of that sideways command then lands in the two weakest directions of L_{xy} , about **130×** below the strongest. Those two directions are an **orbit about the target** ($v_x = -Z \cdot \omega_y$, $v_y = +Z \cdot \omega_x$) — the motion that barely changes the image, which is exactly why they are weak and exactly why the pseudo-inverse amplifies them.

The camera swings nearly edge-on. Over 100 random starts per angle (lateral ≤ 2 cm, tilt $\leq 3^\circ$ per axis, exact features), at 180°:

	switched	converged	lost target edge-on	furthest	peak tilt m median/max
partition always on	0	97	3	0.80–0.81 m	84° / 89°
σ_6 guard	0	97	3	0.80–0.81 m	84° / 89°
area guard	100	100	0	2.02–2.88 m	76° / 78°

Read that carefully, because it is a trade and not a verdict:

- The **area guard** fires in all 100 runs. It is wrong about *why* the view is shrinking, and the law it hands control to is the one that retreats, so the camera backs off to 2–2.9 m instead of 0.80 m. But it never lost the target.
- σ_6 is right that the target and the matrix are healthy, so it never switches — and therefore inherits every orbit, including the three that go fully edge-on and lose the target.

Calibration cannot buy both. Widening the healthy tilt range the threshold is drawn from shrinks the retreat (2.59 m $\rightarrow$ 1.26 m at 0.9 rad) until, at 1.2 rad, the guard stops firing at all and loses the target alongside the partition — and the same widening blinds it further to a mild collapse (1.19 $\times$ $\rightarrow$ 2.46 $\times$).

And 0.5 px of detector noise is enough to trigger the whole thing from the exact start: the area guard drops the partition for 123–125 steps and backs away to 1.80–2.03 m across five noise seeds, while σ_6 does not switch.

What changed in the repo because of this

- Dead claim 4 was rewritten twice, with both tables.
- The results table gained a note that its retreat start is an **exact** symmetric special case, and what happens from an imperfect one.
- The noise paragraph now says the steadier *statistic* did not produce the steadier *decision*, with both numbers.
- Test sections 20 and 21 lock all of it, including the orbit mechanism: that 100% of the commanded sideways motion lies in the two weak directions, and that those directions are the orbit at the target's depth.

Say it like this

Putting it on the arm found two things the free-flying study could not. One: the guard fires on a healthy part seen at a steep angle, because obliquity and degeneracy shrink the marker patch identically — and the spectral signal moves the other way on the same event, which is what revived a claim I had killed. Two: the benchmark that made the guard look best starts from an exactly symmetric pose, and from a start 2 cm off, the partition's own solve lands in a direction the image can barely see and swings the camera edge-on. The guard catches that and pays for it with a two-metre retreat; the spectral rule stays silent and loses the target in 3 runs out of 100. Neither dominates, and the README says so.

26. The folding part, and the clip

In plain words

After the tilting panel turned out to be a false alarm, the demo needed a case where the 2001 controller genuinely cannot cope — not a trick, a real one.

Here it is. The panel is split into two flaps on a hinge, like a book closing. Mid-run both flaps fold away from the robot, so the six markers **physically collapse toward a line in space** while every one of them stays in view and perfectly detected. Nothing is occluded. Nothing is blurry. The part simply stops being flat.

The 2001 controller judges distance by how big the marker patch looks. The patch shrinks, so it concludes "I am too far away" and drives at the part. The guarded arm notices the patch is outside its healthy range and switches to the plain law, which doesn't use the patch at all, and holds its ground.

Designing the scenario honestly

The rule I set myself: **gate it in numpy first**. If the unguarded law does not clearly misbehave and the guarded one clearly hold, there is no clip, because staging a scenario until the demo looks good is how demos start lying.

The gate used the arm's own control law and measured, for folds from 70° to 85° at speeds from 0.5 s to 3 s:

- the unguarded partition lunges to **2-8 cm** and loses the markers, at every fold tested;
- **plain IBVS holds** 20-22 cm at every fold tested, which proves the partition is what fails, not the arm or the detector;
- **σ_6 never fires** — minimum 1.29× — because folding away from the camera adds depth variation and the matrix stays well conditioned;
- the **area guard fires only inside a speed window**.

That last point is the most important thing in this chapter, and it is a limitation, not a feature. The guard watches the polygon area. The partition's lunge **restores** that area by getting closer. So a fold slow enough for the control loop to absorb never crosses the threshold: at 85° over 3 s the area never drops below 1.16× and the guarded cell fails exactly like the unguarded one. Fast enough — 75° within 1 s, 80° within 1.5 s, 85° within 2 s — and the collapse outruns the loop and the guard fires.

A guard on a quantity its own controller regulates is only as good as that race.

That sentence is now in the README's limits.

The clip uses 85° in 1 s, comfortably inside the window, and the repo states the window.

Building it in Gazebo

Design choices that each fixed a specific problem:

- **Markers are spheres, not discs.** A disc on a flap that has rotated 85° is seen edge-on and nearly vanishes; a sphere projects as a circle from any angle, so the detector keeps working through the whole fold.
- **Sphere centres sit on the flap's mid-plane.** If the markers stand proud of the surface, the fold cannot bring their centres toward the hinge line, and the collapse in 3D is only partial.
- **The marker ring is rotated 11° .** At 0° two pairs of markers share the same column, so they merge into one blob as the pattern flattens and the detector drops to five features. 11° keeps every column distinct.
- **A light backdrop sits 11 cm behind the hinge,** so a dark marker never lands on a dark background as the flaps swing back.
- **The flaps fold away from the arm,** so the arm can never touch them and the physics stays clean.
- **Both cells run the same protective stop:** no commanded motion may bring the camera within 12 cm of the hinge plane, checked on the *next* pose. Identical rule on both sides, so the comparison stays fair. This is also what keeps the unguarded failure realistic rather than a simulated crash.

What one run measured

Both cells are identical until the fold. Then:

	partition always on	with the guard
guard decision	not used	fires 0.96 s into the fold, margin 0.90×
standoff	26.3 cm → protective stop at 16.2 cm, 1.46 s in	holds 22-24 cm
markers detected	6/6 throughout	6/6 throughout

And the same act in numpy with the same law: the partition reaches 6.8 cm and loses the markers for 38 steps; the guard fires 0.9 s in and holds 18 cm; plain IBVS holds 22.5 cm. Locked as `tests.py` section 22.

What the act deliberately does not show

Two things were measured and are **not** in the clip, both in the README's limits:

- **The fallback creeps.** Held folded for several seconds, the plain law's commanded speed grew from 0.03 to 0.60 over 4 s and the marker correspondence started flipping. A fully folded part leaves an orbit about the hinge line that the image cannot see — the same weak-direction story as chapter 25 - and the controller drifts along it.
- **Re-engaging the partition mid-unfold is violent.** When the flaps opened again, the guard re-enabled the partition while the part was still folded to 62° , the first command after that was **6.05**, and the target was lost within a second.

So the act ends 2.6 s after the fold, and the clip claims only what the act shows. The guard decides *which* law to run; it does not make a law for geometry that is genuinely unobservable.

How the clip was made

1. `run_fold_demo.sh` with `RECORD=1` runs the act and saves **every frame** from two film cameras (60 Hz) and both wrist cameras (20 Hz), each named by its simulation timestamp, plus a 10 Hz CSV of everything the controller knew.
2. Gazebo is slowed to about a quarter of real time so the 1080p cameras never drop a frame. Because every node runs on simulation time, this changes nothing about the control loop.
3. `arm_clip.py` cuts the clip from those frames and that log: hook, title, approach, the fold in slow motion with live wrist views and the guard gauge, then the logged distance curves, then two end cards.

The rules I held myself to, which are written at the top of `arm_clip.py`:

- every frame of robot motion is a frame Gazebo rendered during that run;
- every number on screen is read from that run's log;
- the edit changes only timing, crops and overlays — no re-staging, no re-enactment, no numbers typed in by hand;
- the end card states the limits rather than hiding them.

A produced clip is fine. A fabricated one would make every number in the repo worthless, which is the actual asset.

Say it like this

The tilting panel was a false alarm, so I built the case the 2001 law genuinely cannot survive: a part that folds, so the markers collapse toward a line in 3D while all six stay perfectly visible. The controller reads the shrinking pattern as distance and drives at the part; it trips the cell's protective stop at 16 cm. The guarded arm switches a second into the fold and holds 22. I gated the whole scenario in numpy before filming it, and the clip's end card says the part the demo can't show: the guard only catches folds faster than the control loop can absorb.

Part 6

Using it

27. Limits, collected

Every limit the project found, in one place. This chapter exists because a result without its boundary is advertising.

About the guard itself

It only wins a race. The guard watches the polygon area — and that is the same area the partition drives to its goal. So a degeneracy slow enough for the control loop to absorb never crosses the threshold. Measured on the folding part: 85° over 3 s is missed completely and the guarded cell fails exactly like the unguarded one. It fires at 75° within 1 s, 80° within 1.5 s, 85° within 2 s.

It cannot tell obliquity from degeneracy. A healthy planar part viewed at 70° fires the guard at 0.99× while σ_6 reads 27.01× and is right. On the retreat benchmark from an imperfect start that false positive costs a 2.02–2.88 m retreat in 100 runs out of 100 (chapter 25).

Calibration cannot buy both. Widening the healthy tilt range shrinks the retreat (2.59 m → 1.26 m at 0.9 rad) until, at 1.2 rad, the guard stops firing and loses the target alongside the partition — and the same widening blinds it further to a mild collapse (1.19× → 2.46×).

Beyond it there is still no controller for a collapsed target. Held folded, the fallback creeps along an orbit about the hinge line that the image cannot see ($\|v\|$ 0.03 → 0.60 over 4 s), and re-enabling the partition mid-unfold produced a 6.05 command spike that lost the target. The guard decides *which* law to run; it does not make a law for geometry that carries no information.

Three configurations no controller finishes. Permanently two markers, three permanently collinear points, and a goal on the three-point danger cylinder. In all three, classic IBVS, truncation and the switched controller land on the identical pose error to four decimal places (0.0695 m, 0.1130 m). The residual is information the features never carried.

About the thresholds

They do not transfer between targets. Healthy 1st percentile: 3.87e-3 for the ring, 1.74e-2 for the square, 4.5× apart. Calibration is per target.

One margin is structurally thin. With a single 6-marker threshold the dropout case clears by only 1.19×, because a subset's polygon is bounded by the full set's. Conditioning on the visible count restores 45.9×. That is a fix for a defect, not a free property.

Calibrate on a known-good part, never in situ. In-situ calibration on the collapsed target gives a threshold 7× lower (7.96e-2 → 1.13e-2), after which the guard never fires and everything still looks like it is working.

About the constants

The shipped τ has a constructible case where it is worse than no intervention.

On a collapsed target with an initial rotation about the collapse line, the switched controller at $\tau=1e-3$ stalls at a pose error of 0.297 m while plain classic IBVS converges exactly. $\tau=1e-5$ completes it. It is a tuning failure, not a structural one, but the default has such a case and τ is a single global constant chosen from one geometry. Sweep it on yours.

Clean cost is threshold-dependent. +13% at a 1% convergence threshold, +7.4% at 0.01%, +2.4% at 50%. Meaningless quoted without its threshold.

About the evidence

The core study uses exact feature positions. A free-flying virtual camera, ground-truth features, no detector, no correspondence errors, fixed $dt = 0.033$ s, no actuator dynamics, no joint limits, no contact. This is the single largest open question, and it is why the rendered-camera port and the Gazebo arm exist.

Two targets only. A six-point ring and a four-point square.

No hardware anywhere. Rendered cameras and simulated arms are not a real cell: no lens distortion beyond the model, no rolling shutter, no thermal drift, no vibration, no real lighting.

σ_6 's only demonstrated edge over the area guard is measure-zero. At the three-point danger cylinder $\sigma_6 = 2.2e-16$ while the polygon area is constant to $1.5e-16$ relative, so the area guard cannot fire even in principle — but no sustained trajectory dwells there, and with the goal on the cylinder every controller floors at about $7e-4$ with no winner. A detection difference that produces no control difference is not a result.

With fiducial markers the collapsed case is structurally untestable. The degeneracy is the absence of 2D extent and an ArUco marker needs 2D extent to decode: the verdict needs $c \leq 0.05$, and the smallest collapse leaving six decodable markers at this distance is $c = 0.35$, even at 2560 px wide.

Correlated noise was tested, and the detector fails first. Motion blur driven by the camera's own inter-frame motion: the guard's margin is flat right up to the cliff ($1.62\times$ at 0, 2.5, 5.1 px of blur) and then the detector fails outright at 12.7 px, which is about a quarter of a marker width. Correlated degradation does not bias the guard, it removes its input — so a guard failure and a detection failure stay distinguishable. Still untested: a platform that holds speed through a degradation window, since this controller is decelerating by then.

How to say this in an interview

Volunteer the three that matter before you are asked:

It's simulation, not hardware. The guard watches a quantity its own controller regulates, so a slow collapse is masked — measured, 85° over three seconds is missed entirely. And the shipped truncation constant has a constructible case where intervening is worse than doing nothing.

Volunteering a limit is the cheapest credibility available, and it shifts the conversation from "can I catch this person out" to "what would you do next".

28. Telling the story

The 30-second version

Image-based visual servoing steers a robot straight from what its camera sees. It has a famous failure — asked to rotate 180°, it flies the camera backwards — and the standard 2001 fix for that failure causes two worse ones: with two features left it spikes nearly 300×, and on a collapsing target it can't converge. I made that fix a runtime decision instead of a permanent one, with a one-line check on the health of the feature the controller relies on. It closes the gap from 11.6× to 0.5×, which killed my own hypothesis that the decision needed a learned policy. It ships as a library and a ROS 2 node, and runs closed-loop on a UR5e in Gazebo.

The 2-minute version

Add, in this order:

1. **Why the four failures are different.** Retreat is coupling with a healthy matrix; dropout is conditioning; two-features is a structural failure of one controller; collapse is missing information that feature counting cannot see. One rule has to cover all four, which is why I implemented six candidate rules and compared them instead of picking one.
2. **What the guard watches and why.** The 2001 law uses the area of the marker polygon as a stand-in for distance, so that area is what has to be healthy. The threshold comes from calibration on a known-good part: a few thousand healthy viewpoints, first percentile, per target and per number of markers visible.
3. **The honest ledger.** The area signal can't tell a degenerate part from a healthy one seen at a steep angle. The spectral signal σ_6 can, but it's blind to a folding part. Neither dominates, both failure directions are measured, and the README says so.
4. **How it was verified.** Derivatives by finite difference, not by reasoning about sign conventions — that caught two sign bugs. 177 regression tests, including tests that the things that didn't work still don't.
5. **What it runs on.** A ROS 2 node publishing one boolean, and a Gazebo UR5e cell where the loop is perception → guard → control law → the arm's own Jacobian at 10 Hz on simulation time.

The 10-minute whiteboard version

Draw these five things in order. Each one is a chapter of this book.

1. **The projection and the interaction matrix** (chapter 7). Point at the two rows. Say what each column means. Point out that v_x and ω_y look alike for a small far target — that's the weak direction everything later turns on.
2. **The control law and its inverse** (chapter 8). $v = -\lambda L^+ e$. Say: every failure in this project happens inside this inverse.
3. **The partition** (chapter 14). Cross out two columns; write v_z from area and ω_z from line angle beside them. Say: this fixes retreat and makes the remaining solve exactly

determined, which is the 288× spike.

4. **The switch and the guard** (chapters 16–17). One comparison per step; threshold from calibration; what the margin means.
5. **The two counter-examples** (chapters 25–26). Oblique healthy part: area fires, σ_6 silent. Folding part: area fires correctly, σ_6 silent at 1.29×. Say: same disagreement, opposite verdicts, which is why the repo ships the trade rather than a winner.

Question bank

"Why not use learning?"

I planned to. I set the bar in advance: a hand-written rule had to get the two-feature spike under 2× for the learned plan to be dropped. It reached 0.5×. Training a policy would have meant data collection, a reward, and an uninspectable model, to replace a comparison between two numbers.

"Why the area and not the matrix spectrum?"

Because the area is the quantity the 2001 law actually leans on, and it's cheap — no SVD — and steadier under detector noise: 2 px of per-point noise moves it 0.33% of its threshold against σ_6 's 1.03%. But I don't claim it dominates. On an oblique healthy part it cries wolf, and the cost is a 2 m retreat in 100 runs out of 100. On a folding part it's the only one of the two that fires. Both are measured and both are in the README.

"How do you set the threshold?"

Calibration on a known-good target: thousands of healthy poses around the goal, first percentile, seeded and reproducible, conditioned on how many markers are visible because singular-value interlacing means a subset's statistic is bounded by the full set's. And it must not be calibrated in situ — I measured that: the threshold comes out seven times lower and the guard silently never fires again.

"What happens under noise?"

Independent noise it shrugs off, $d' = 454$ at realistic levels. The hard case is noise correlated with the failure, which is why I built a rendered camera: motion blur driven by the camera's own inter-frame motion. The finding is that correlated degradation doesn't bias the guard, it removes its input — the margin is flat right up to the cliff and then the detector fails outright — so a guard failure and a detection failure stay distinguishable.

"What's the weakest part of this?"

Three things. It's simulation, not hardware. The guard watches a quantity its own controller regulates, so a slow collapse is masked — measured: 85° over 3 s is missed completely. And the shipped truncation constant has a constructible case where intervening is worse than doing nothing: $\tau=1e-3$ stalls at 0.297 m where plain classic IBVS converges exactly.

"Tell me about something you got wrong."

I claimed the guard's switch caused a velocity spike in the arm demo and wrote it down. Then I reproduced it across 16 tilt onset/speed combinations and the peak velocity was identical under all three rules in 15 of them. The spike never reproduced; it was most likely perception noise at a grazing view. I corrected it in the file where the wrong claim was rather than deleting it.

"How do you know your implementation is right?"

Finite differences. A derivative is a prediction about what happens under a small change, so I make the change and compare — 16 checks covering every derivative, sign and null space. Two sign bugs in the partitioned law were found that way and neither would have been caught by reading the code.

"What would you do next?"

Hardware, and a second signal. The two signals fail in opposite directions, so the obvious next step is to run both and report disagreement as its own state, rather than picking one. And I'd sweep τ per geometry instead of inheriting one constant.

ROS-specific questions

"What ROS 2 did you actually use?"

Packages, nodes, topics, parameters, namespaces and remappings; launch files with staged actions; vision_msgs, sensor_msgs, std_msgs, diagnostic_msgs, trajectory_msgs, visualization_msgs; TF; RViz; ros2_control with gz_ros2_control and a JointTrajectoryController per arm; ros_gz_bridge for clock, commands and images; and simulation time everywhere.

"How does the guard node fit into someone else's stack?"

It subscribes to vision_msgs/Detection2DArray and sensor_msgs/CameraInfo and publishes std_msgs/Bool plus a diagnostic. It doesn't control anything, which is deliberate — you keep your servo loop and gate it on the Bool, so it can be adopted without rewriting a controller.

"Why does detection ordering matter?"

The signal is a shoelace polygon area, so it depends on point order, and a detector returns points in whatever order it found them. A self-crossing order gives a small area and fires the guard for a reason unrelated to geometry. So detections are ordered by marker id, with an angular fallback, the ordering used is published in the diagnostic, and the node cross-checks the shoelace area against the convex hull and escalates to ERROR if they disagree.

"How do you handle time in simulation?"

Every node runs with use_sim_time off /clock. It matters here: recording 1080p cameras drops Gazebo to about a quarter of real time, and on sim time that changes only wall-clock duration. An earlier version used a wall-clock timer, so turning recording on silently changed the control rate.

"How is the arm driven?"

The control law produces a camera twist. I map it through the arm's geometric Jacobian — hand-written numpy from the URDF, verified against TF to $1e-6$ m — clip joint velocities to ± 1.5 rad/s, integrate one step, and publish a single-point JointTrajectory to the JointTrajectoryController with a 0.16 s horizon at 10 Hz.

"What broke while you were building it?"

Pick two from chapter 23. The good pair: the detector finding seven blobs because the robot's own wrist enters the frame at close standoff — fixed by selecting the six most similar in area rather than the six largest; and ordering six identical dots by angle fixing their order but not where the order starts, so one cell converged to 0.090 while its twin diverged to 0.530 running identical code.

How to describe the ROS part accurately

Say it this way, because it is both stronger and checkable:

*The control study is numpy, deliberately — it's where the measurements are exact and reproducible. The **deployment** is ROS 2: the guard ships as a node, and the robot demonstration is a full ROS 2 + Gazebo cell with ros2_control, closed loop through the guard.*

Do not say the entire project runs on ROS. Anyone who opens the repo sees a numpy study at its core, and being caught rounding up on a small thing makes every number you quote suspect. The split is a strength: the study is exactly reproducible because it is not entangled with a simulator, and the same control law is then imported by the ROS node so the two cannot drift apart.

Glossary

term	meaning
IBVS	image-based visual servoing: control directly from image error
feature	a point the detector finds — here, a marker centre
interaction matrix L	maps camera velocity to image-feature velocity
pseudo-inverse	least-squares "inverse" of a non-square matrix
singular value	how strongly the matrix responds in one direction
σ_6	the 6th singular value, padded — zero if the matrix has a null space
truncation (τ)	skipping directions weaker than $\tau \times$ the strongest
the partition	the 2001 scheme driving v_z and ω_z from image measurements
the guard	the runtime check on the partition's own area feature
margin	signal $\div$ threshold; below 1 means the guard fires
node / topic	a ROS 2 program / a named channel it publishes or subscribes to

term	meaning
ros2_control	the ROS 2 framework that owns hardware interfaces and controllers
TF	the ROS transform tree: where every frame is relative to every other
sim time	clock published by the simulator, so nodes run on its schedule

Last thing to say

If the conversation ends and you only get one sentence, use this one:

The result I'm proudest of is the one that killed my own plan, and the repository documents seven of those with the numbers that killed them.

29. Glossary, and every number in one place

Glossary

term	meaning
IBVS	image-based visual servoing: control computed directly from image error
PBVS	position-based: estimate the 3D pose first, then control it
feature	a point the detector finds; here, a marker centre
normalised image coordinates	pixels with the intrinsics divided out: $x = (u - c_x)/f_x = X/Z$
intrinsics	focal lengths and principal point; arrive as <code>camera_info</code> in ROS
principal point	where the optical axis meets the sensor; $(W-1)/2$, not $W/2$
frame	an origin plus three axes; every coordinate is relative to one
pose	position and orientation together, kept as a 4×4 matrix
twist	six numbers: three linear and three angular velocities
se3_exp / se3_log	apply a twist to a pose / recover a twist from a pose difference
interaction matrix L	maps camera twist to feature velocity in the image, $2N \times 6$
pseudo-inverse L^+	least-squares inverse; minimum-norm when under-determined
SVD	rotate, stretch along six axes, rotate: $L = U \Sigma V^T$
singular value	how strongly the matrix responds along one direction
σ_6	the sixth singular value, padded — exactly 0 if L has a null space
null space	directions the matrix cannot see at all
ill-conditioned	some direction is nearly invisible, so the inverse amplifies
rank	how many directions the matrix can actually see
truncation (τ)	skipping directions weaker than $\tau \times$ the strongest
the partition	the 2001 scheme: v_z from polygon area, ω_z from a line angle
coupling term	$L_z v_z$ added so the other four DOF cancel what the partitioned two cause

term	meaning
the guard	the runtime check on the partition's own area feature
margin	signal ÷ threshold; below 1 means the guard fires
calibration	the healthy pose distribution that sets the threshold
spike	peak commanded $\ v\ $ during a disruption ÷ the same controller's $\ v\ $ before it
convergence	final image error below 1e-4
clean cost	extra steps on an undisturbed run, at a stated convergence threshold
node / topic / message	a ROS program / a named channel / the typed thing on it
ros2_control	the ROS 2 joint-level control framework
TF	the live tree of transforms between frames
sim time	the simulator's clock, published on <code>/clock</code>
danger cylinder	the surface through three points where the geometry is singular

Every number, and where it comes from

The four failures

number	meaning
68.19 m	how far classic IBVS retreats on a 180° rotation
−1.0000	command/error alignment during that retreat — perfect, while failing
13.77 m	truncation on the same case: still diverges
0.80 m	every partition-based controller: never backed up
28.7×	classic IBVS spike on 3-of-6 dropout
0.9×	truncation on the same dropout
288.4×	partitioned spike with two features
11.6×	the combined controller on two features — the gap to close
0.5×	every switched rule on two features — the gap closed
7.3e-3 / 2.0e-3	partitioned / combined floors on the collapsed target
2.4e-5	truncation and the geometric guards: converged
[3.68, 3.66, 0.0124, 0.0037]	the reduced matrix's singular values when collapsed

The guard

number	meaning
1st percentile, 4000 poses, seed 0	the calibration procedure
$3.87\text{e-}3$ / $1.74\text{e-}2$	healthy threshold for the ring / the square, 4.5× apart
$1.19\times \rightarrow 45.9\times$	dropout margin, single threshold vs conditioned on visible count
$7.96\text{e-}2 \rightarrow 1.13\text{e-}2$	known-good vs in-situ threshold: 7× lower, guard disabled
0.33% / 0.76%	how far 2 px of noise moves the area statistic (square / ring)
1.03% / 2.35%	the same for σ_6 : about 3× more
$0.5\times \rightarrow 3.0\times \rightarrow 3.7\times \rightarrow 5.7\times$	two-feature spike with hysteresis of 0, 2, 3, 5 steps
+13%	clean cost of the partition at a 1% convergence threshold

Dead claims

number	what it killed
$0.5\times$ vs a 2× bar	the learned-policy plan
rank 3 in a 4×4	the rank-margin rule
$1.65\text{--}3.73\times$ overlap band	a single global spectral threshold
$27.01\times$ vs $0.99\times$	claim 4's revival: σ_6 silent where area fires
98.78% of $\sigma_{\min}$	the collapsed-target spike, as τ artifact
231 ms / 0 ms / 0 ms	forward prediction: warning only where it is not needed
15 of 16 identical	my retracted claim that switching caused an arm spike
$9.97\text{e-}2$ vs exactly 0	why the health metric had to be padded σ_6

The rendered camera

number	meaning
0.095 px	rendered-and-detected vs exact projection: the gate
$(-0.4904, -0.5010)$ px	the half-pixel principal-point bias
$0.1266 \rightarrow 0.5972$ px	APRILTAG refinement before / after fixing that bias
0 of 6 $\rightarrow$ 6 of 6	marker detection before / after mirroring the texture
$254 \rightarrow 13$	image mean when the auto-computed extent clipped the scene
156 px	how far a ghost from the blur model displaced a feature
$d' = 454$	separation of healthy and degenerate under realistic noise

number	meaning
1.62× flat, then 50% detector failure at 12.7 px	correlated degradation: flat, then cliff
$c \leq 0.05$ needed vs $c = 0.35$ achievable	why markers cannot test the collapsed case

The arm

number	meaning
1e-6 m	hand-written forward kinematics vs TF
2.1e-15	the partition's sideways command at the exactly symmetric retreat start
~130×	amplification of the weak orbit directions of <code>L_xy</code>
97 / 3 of 100	partition and σ_6 at 180° from imperfect starts: converged / lost target
100 / 0 of 100	area guard on the same starts: converged / lost target
2.02–2.88 m	how far the area guard's false positive backs the camera away
123–125 steps, 1.80–2.03 m	the same triggered by 0.5 px of noise, five seeds
26.3 → 16.2 cm, 1.46 s	the unguarded arm's lunge into the protective stop, folding part
0.96 s, 0.90×	when the guard fired, and its margin
22–24 cm	where the guarded arm held
6.8 cm / 18 cm / 22.5 cm	the same act in numpy: partition / guarded / plain IBVS
1.29×	σ_6 's minimum during the fold: never fired
1.16× at 85° over 3 s	the masked slow fold: guard never fires

Test counts

number	meaning
177	regression checks in <code>tests.py</code>
16	finite-difference checks in <code>fd_check.py</code>
34	API and ROS-bridge checks in <code>test_blindspot.py</code>

References

- Chaumette & Hutchinson, *Visual servo control part I: basic approaches*, 2006
- Corke & Hutchinson, *A new partitioned approach to image-based visual servo control*, IEEE T-RA, 2001
- Malis, Chaumette & Boudet, *2½D visual servoing*, 1999
- Michel & Rives, *Singularities in the determination of the situation of a robot effector from the perspective view of three points*, INRIA, 1993